

You Only Decapsulate Once: Ciphertext-Independent Single-Trace Passive Side-Channel Attacks on HQC

Zhenzhi Lai*, Ruiyi Zhang[†], Zhiyuan Zhang^{‡§}, Julius Hermelink[‡],
Michael Schwarz[†], Van-Thuan Pham* and Udaya Parampalli*

*The University of Melbourne, Australia

[†]CISPA Helmholtz Center for Information Security, Germany

[‡]Max Planck Institute for Security and Privacy, Germany

Abstract—Hamming Quasi-Cyclic (HQC) has recently been selected by NIST, after the Round 4 submission, as a post-quantum key encapsulation mechanism (KEM) standard and will soon be widely deployed. Therefore, it is important to ensure its implementation is constant-time, i.e., resistant to side-channel attacks. Existing timing attacks on HQC exploit non-constant-time source code and the decryption that is vulnerable to chosen-ciphertext attacks. These active attacks require constructing thousands of invalid ciphertexts, and thus, they can be easily detected. The latest HQC implementation has mitigated all these attacks by making its source code constant-time.

In this work, we provide a new perspective on reviewing the implementation of HQC and exploiting timing leakages. For the first time, we show that an attacker can recover the secret key of HQC without targeting the CCA-insecure decryption and internal states of message decryption. Specifically, an attacker can exploit the timing leakages that occur when processing sparse vectors, which are ciphertext-independent, to recover the secret key by measuring the leakages only once. We find two such timing leakages in the latest stable HQC implementation, supposedly constant-time, and practically extract the leakages even when the process is protected by AMD Secure Encryption Virtualization. We also show that a power side-channel can extract similar leakages on embedded devices.

Our findings apply to all code-based KEMs that are submitted to the NIST Round 4 PQC submission. We show that an attacker can also perform similar passive attacks to recover the session key of BIKE and Classic McEliece. To help write constant-time code, we propose and test a workflow that uses CT-grind when developing the code. We find that CT-grind can effectively find all timing leakages in various implementations of HQC. Therefore, we suggest that cryptographic developers constantly use constant-time analysis tools when developing code.

1. Introduction

The advent of quantum computers poses a fundamental threat to public-key cryptosystems such as RSA and ECC, as Shor’s algorithm [1] solves the underlying integer factorization and discrete logarithm problems in polynomial time.

The harvest-now, decrypt-later strategy calls for immediate action to replace quantum-insecure cryptographic algorithms with quantum-resistant ones. This new generation of cryptography is referred to as post-quantum cryptography (PQC).

NIST started the PQC standardization process in 2016 and selected the Module-Lattice-based Key-Encapsulation Mechanism (ML-KEM) [2] as the first Key Encapsulation Mechanism (KEM) after the Round 3 submission in 2022 [3]. To diversify the standardized KEMs, NIST started the Round 4 submission to evaluate Hamming Quasi-Cyclic (HQC) [4], Bit Flipping Key Encapsulation (BIKE) [5], Classic McEliece [6], and Supersingular Isogeny Key Encapsulation (SIKE) [7]. Except for SIKE, which has been found to be mathematically insecure [8], all three remaining candidates are code-based KEMs and are believed to be algorithmically secure [9]. Recently, NIST has announced that HQC is the only KEM to be standardized after the fourth round [9]. Although the other two submitted KEMs are not to be standardized immediately, they are still under consideration by NIST for future standardization [9].

Besides being secure at the algorithmic level, it is also important to ensure that the implementation is secure [10]. As noted by NIST, a scheme is more desirable if it mitigates side-channel attacks with minimal performance overhead [11]. One important principle to protect against side-channel attacks, specifically timing side-channel attacks [12], is the constant-time programming policy [13]. It disallows secret-dependent control-flow transfers, memory accesses, and variable-time operations (e.g., division). Not enforcing constant-time programming before widely deploying a scheme may result in serious vulnerabilities. A recent example is that the reference implementation of ML-KEM, which has already been widely deployed, is found to be vulnerable to timing attacks where an attacker can recover the long-term secret key [14]. A large number of libraries were affected and thus had to be patched [15]. With the announcement of standardization, HQC will soon be widely deployed in common cryptographic libraries. Therefore, it is important to ensure its implementation is constant-time before the wide adoption.

The implementations of HQC, submitted to Round 3 and Round 4 NIST PQC selection, have been evaluated by several works [16], [17], [18]. These attacks all exploit the

[§] Corresponding author.

timing leakages in the source code and recover the long-term secret key with the known fact that HQC Public-Key Encryption (PKE) is not Chosen-Ciphertext Attack (CCA) secure. Specifically, they construct (tens of) thousands of invalid ciphertexts and send them to the decapsulation routine. They measure the timing or micro-architectural differences for these ciphertexts to obtain information on the decrypted message and thereby on the secret key. These attacks require the attacker to submit a large number of chosen ciphertexts and can be easily detected. The latter is because decryption failures—caused by the chosen ciphertext—happen very rarely in practice (with a probability of less than 2^{-128} [4]). After the publication of these attacks, the HQC design team and the most popular PQC library, PQClean [19], took quick action to update their implementations. Therefore, the latest stable implementation of HQC, provided by PQClean, is secure against all known timing attacks [16], [17], [18].

In this paper, we provide a new perspective on reviewing and exploiting insecure HQC implementations and extend it to all code-based KEMs submitted to the NIST R4 PQC submission. For the first time, we show that an attacker does not need to target the CCA-insecure decryption to recover the long-term secret key, which previous timing (and power) side-channel attacks on HQC rely on [16], [17], [18], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31]. Instead, we show that partial secret key bits can be recovered independent of the ciphertext in a single trace. Therefore, an attacker can passively collect side-channel information without controlling the ciphertext sent to the victim’s decapsulation process. We identify two new timing leakages in HQC and demonstrate two ciphertext-independent single-trace passive side-channel attacks to recover the long-term secret key. The first attack is referred to as *Sparse Vector Hamming Weight Attack*, which exploits the timing leakages that occur when writing 1s into a key vector during the key re-sampling process. We leak the HW of each key word. Since the secret key is two sparse vectors, the HW of most of their words would be zero. Using Gaussian Elimination [18], [32] effectively recovers the remaining bits. We refer to the second attack as the *Vector Multiplication Attack*. We directly recover around 94% secret key bits by monitoring the vector multiplication, which reveals 60 bits per word (i.e., 64-bit). The remaining bits can be either recovered with Information-Set Decoding (ISD) [33] or even with brute-force. To the best of our knowledge, the two attacks are the first ciphertext-independent side-channel attack and purely passive side-channel attack on HQC.

We target the latest stable implementation of HQC, provided by PQClean, which mitigates all known HQC timing leakages [16], [17], [18] and is supposed to be constant-time. The source code of this implementation is indeed constant-time. However, we find that common compiler optimizations (e.g., `-O2`) may generate variable-time binary. We conduct two end-to-end side-channel attacks and show that an attacker can automatically extract all secret-related side-channel information even when the decapsulation process is protected by AMD Secure Virtualization Encryption (SEV). Using the FARFETCH’D framework [34], an attacker only

needs to perform the attack once with an arbitrary ciphertext. In addition to the specific timing leakages, we also show that similar leakages can be extracted via a power side-channel attack.

Our findings are not limited to HQC. In fact, we show that the implementation of all code-based KEMs submitted to the NIST PQC R4 submission is vulnerable to our first attack. Specifically, BIKE and Classic McEliece use similar implementations to write 1s to error vectors. Both their source codes are constant-time, but compiler optimizations induce vulnerabilities. Although an attacker cannot exploit them to recover the long-term secret key, as we have done to HQC, an attacker can still recover their session keys. We conduct proof-of-concept attacks to confirm that all required side-channel information can be precisely extracted by an attacker.

Our findings reveal a common side-channel vulnerability existing in code-based KEMs, which had not been identified by prior work. These vulnerabilities arise under specific but common compiler versions and optimization flags and may therefore be difficult to detect. We propose and validate a constant-time analysis workflow, based on CT-grind [35], to mitigate timing leakages during code development. We demonstrate that this four-step process, when properly configured, detects all known timing leakages in HQC (including those exploited by us and prior work [16], [17], [18]). Given by the fact that subtle timing leakages can lead to powerful attacks, as shown by our attacks, we recommend that cryptographic code developers constantly use constant-time analysis tools when writing code.

To summarize, we make the following contributions:

- 1) For the first time, we show that timing side-channel attacks on HQC do not require chosen ciphertexts. We present the first two ciphertext-independent single-trace passive side-channel attacks on HQC. We refer to the first attack as *Sparse Vector Hamming Weight Attack* and the second attack as *Vector Multiplication Attack*. We further present the first single-trace ciphertext-independent power side-channel attack against HQC.
- 2) We demonstrate that both timing side-channel and power side-channel can be exploited to carry out the *Sparse Vector Hamming Weight Attack* on HQC.
- 3) We show that all code-based KEMs submitted to NIST Round 4 PQC submission are vulnerable to the *Sparse Vector Hamming Weight Attack*. An attacker can recover the long-term secret key of HQC and the session keys of BIKE and Classic McEliece through a single-trace passive side-channel attack.
- 4) We propose and evaluate a constant-time analysis workflow to detect and mitigate timing side-channels when developing cryptographic code. We show that it can effectively detect all known timing attacks on HQC with low analysis overhead.

Responsible Disclosure. We have reported the two timing vulnerabilities in HQC to the PQClean team on 9 June 2025. The PQClean team forwarded our report to the HQC team

on the same day. Shortly, we received a response from the HQC team, who acknowledged that a fix is in the pipeline. The HQC team releases a newer version of HQC on 22 Aug 2025. We confirm that the timing leakage in the vector multiplication is fixed. However, the timing leakage in the key re-sampling is not completely fixed. In Section 6.1, we explain how to exploit the remaining leakage. We have sent this paper to PQClean, HQC, BIKE, and Classic McEliece developers on 13 Nov 2025. On 14 Nov 2025, we received a response from the PQClean team, who forwarded our report to other relevant developers, and a direct confirmation from the HQC team that fixes will be provided. On 20 Nov 2025, the Classic McEliece team informed us that the Classic McEliece implementation in PQClean is not maintained. After receiving our report, they had updated their official website and pointed the latest official implementation to libmceliece, which detects constant-time violations with CT-grind.

2. Background

In this section, we first introduce the notation and background of HQC and then introduce prior side-channel attacks on HQC. Finally, we give an overview of AMD SEV. **Notations.** We denote vectors in bold letters: matrices are denoted in upper-case bold vectors (e.g., $\mathbf{A}$), while ring elements are denoted in lower-case letters (e.g., $\mathbf{a}$). Note that often the same element may be seen as either a vector or an element of a ring. We denote the vector multiplications as $\mathbf{a} \cdot \mathbf{b}$. Finite fields with $q = p^n$ elements, where p is a prime, are denoted by $\mathbb{F}_q$; $\mathbb{F}_2$ denotes the binary field with two elements. The Hamming weight (HW) of a binary vector is defined as the number of non-zero entries in the vector.

2.1. HQC

HQC [4] is a code-based KEM that has been selected for standardization in the NIST post-quantum standardization process [9], [36]. It aims to establish a shared secret, known as the shared session key $\mathbf{ss}$, between two parties using public-key cryptography. Its security is based on the hardness of the Quasi-Cyclic Syndrome Decoding (QCSD) problem.

2.1.1. HQC PKE and KEM. HQC is built by deriving a Indistinguishability under Chosen-Ciphertext Attack (IND-CCA) secure KEM from an internal Indistinguishability under Chosen-Plaintext Attack (IND-CPA) secure PKE using an Fujisaki-Okamoto (FO)-transform [37], [38], [39]. The PKE defines operations of *key generation* (PKE.Keygen), *encryption* (PKE.Encrypt), and *decryption* (PKE.Decrypt). Then the KEM derives *key generation* (KEM.Keygen), *encapsulation* (KEM.Encaps), and *decapsulation* (KEM.Decaps) from them. We demonstrate those algorithms in Algorithm 1, Algorithm 2, and Algorithm 3. Relevant parameters are presented in Table 1.

The HQC works over the ring $\mathcal{R} = \mathbb{F}_2[x]/(x^n - 1)$ and relies on an Extended Output Function xof and three hash

Algorithm 1 KEM.Keygen (Simplified)

Output: $\mathbf{pk}, \mathbf{sk}$

```

1:  $\triangleright \mathbf{PKE.Keygen}()$ :
2:    $\mathbf{seed}_{\mathbf{sk}} \leftarrow \text{Random}()$ 
3:    $\sigma \leftarrow \text{Random}()$ 
4:    $\mathbf{seed}_{\mathbf{pk}} \leftarrow \text{Random}()$ 
5:    $\mathbf{ctx}_{\mathbf{sk}} \leftarrow \text{XOF.init}(\mathbf{seed}_{\mathbf{sk}})$ 
6:    $\mathbf{x} \leftarrow \text{SampleFixedWeight}_{\mathbf{s}}(\mathbf{ctx}_{\mathbf{sk}}, \omega)$ 
7:    $\mathbf{y} \leftarrow \text{SampleFixedWeight}_{\mathbf{s}}(\mathbf{ctx}_{\mathbf{sk}}, \omega)$ 
8:    $\mathbf{ctx}_{\mathbf{pk}} \leftarrow \text{XOF.init}(\mathbf{seed}_{\mathbf{pk}})$ 
9:    $\mathbf{h} \leftarrow \text{SampleVec}(\mathbf{ctx}_{\mathbf{pk}})$ 
10:   $\mathbf{s} \leftarrow \mathbf{x} + \mathbf{h} \cdot \mathbf{y}$ 
11:  return  $\mathbf{pk} = (\mathbf{seed}_{\mathbf{pk}}, \mathbf{s}), \mathbf{sk} = (\mathbf{seed}_{\mathbf{sk}}, \sigma, \mathbf{pk})$ 
12: return  $\mathbf{pk}, \mathbf{sk}$ 
```

functions $\mathcal{G}$, $\mathcal{H}$, and $\mathcal{K}$. During the key generation phase, the private key $\mathbf{x}, \mathbf{y} \in \mathcal{R}$ of size N for each, also known as the long-term secret key, is sampled by a seed $\mathbf{seed}_{\mathbf{sk}}$. To obtain the public key $(\mathbf{h}, \mathbf{s})$, it samples $\mathbf{h} \in \mathcal{R}$ by $\mathbf{seed}_{\mathbf{pk}}$ and then computes $\mathbf{s}$ by $\mathbf{s} = \mathbf{x} + \mathbf{h} \cdot \mathbf{y}$. Instead of storing those vectors directly, HQC stores the $\mathbf{seed}_{\mathbf{sk}}$ for $(\mathbf{x}, \mathbf{y})$ and $\mathbf{seed}_{\mathbf{pk}}$ for $\mathbf{h}$. As soon as those vectors are used, they are temporarily recomputed from the seeds. Additionally, the random value σ is stored as part of the long-term secret key but is only used to reject an invalid decryption output.

HQC relies on an error-correcting code to correct errors in the decryption. We denote the code used by HQC by $\mathcal{C}$, the generator matrix of the code by $\mathbf{G}$, and the encoding and decoding as $\mathcal{C}.\text{encode}$ and $\mathcal{C}.\text{decode}$.

The KEM.Encaps randomly generates the message m and a randomness salt , where the shared session key is derived from $(m, \text{salt}, \mathbf{pk})$. To encrypt m , the PKE samples $\mathbf{r}_1, \mathbf{r}_2, \mathbf{e} \in \mathcal{R}$ with small HWs w_r, w_e , respectively. The ciphertext $(\mathbf{u}, \mathbf{v})$ is computed as $\mathbf{u} = \mathbf{r}_1 + \mathbf{r}_2 \cdot \mathbf{h}$ and $\mathbf{v} = \mathbf{mG} + \mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e}$. Therefore, during the decryption, a noisy version of the message can be recovered by computing

$$\mathbf{v} - \mathbf{u} \cdot \mathbf{y} = \mathbf{mG} + \mathbf{s} \cdot \mathbf{r}_2 + \mathbf{e} - (\mathbf{r}_1 + \mathbf{h} \cdot \mathbf{r}_2) \cdot \mathbf{y} \quad (1)$$

where the HW of the sum of all terms added to $\mathbf{mG}$ is small in relation to the minimal distance of $\mathcal{C}$ with overwhelming probability. Thus, $\mathcal{C}.\text{decode}(\mathbf{v} - \mathbf{u} \cdot \mathbf{y})$ yields m with a negligible failure rate. Recall that a FO is applied in the KEM. Instead of simply decoding the $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$, KEM.Decaps re-encrypts the decoded message for a validation check and returns a successful or rejected session key accordingly. Such a construction hides the information of the IND-CPA secure PKE.Decrypt result to protect it from a CCA.

Notably, HQC KEM.Decaps only depends on the value of $\mathbf{y}$ but not $\mathbf{x}$. Therefore, recovering $\mathbf{y}$ is sufficient to compromise HQC.

2.1.2. The SDP, Gaussian Elimination and ISD. The attacker may attempt to solve the long-term secret key $(\mathbf{x}, \mathbf{y})$ from the public-key relation $\mathbf{s} = \mathbf{x} + \mathbf{h} \cdot \mathbf{y}$, since $(\mathbf{h}, \mathbf{s})$ are known as the public key. However, the difficulty of solving it is described as solving the Syndrome Decoding Problem (SDP), defined in Definition 1:

TABLE 1. HQC PARAMETERS IN BITS

Instance	RS-S			Duplicated RM			$N_1 N_2$	N	ω	$\omega_e = \omega_r$	$SDP(n, k, t)$		
	N_1	k_1	d_{RS}	Multiplicity	N_2	d_{RM}					$n = 2N$	$k = N$	$t = 2\omega$
HQC-128	46	16	31	3	384	192	17664	17669	66	75	35338	17669	132
HQC-192	56	24	33	5	640	320	35840	35851	100	114	71702	35851	200
HQC-256	90	32	59	5	640	320	57600	57637	131	149	115274	57637	262

Algorithm 2 KEM.Encaps (Simplified)

Input: pk
Output: ss, ct

- 1: $(m, salt) \leftarrow \text{Random}()$
- 2: $(ss, \theta) \leftarrow \mathcal{G}(\mathcal{H}(pk) || m || salt)$
- 3: $\triangleright \text{PKE.Encrypt}(pk, m, \theta):$
- 4: $seed_{pk}, s \leftarrow pk$
- 5: $ctx_{pk} \leftarrow \text{XOF.init}(seed_{pk})$
- 6: $h \leftarrow \text{SampleVec}(ctx_{pk})$
- 7: $ctx_{\theta} \leftarrow \text{XOF.init}(seed_{\theta})$
- 8: $r_2 \leftarrow \text{SampleFixedWeight}(ctx_{\theta}, \omega_r)$
- 9: $e \leftarrow \text{SampleFixedWeight}(ctx_{\theta}, \omega_e)$
- 10: $r_1 \leftarrow \text{SampleFixedWeight}(ctx_{\theta}, \omega_r)$
- 11: $u \leftarrow r_1 + h \cdot r_2$
- 12: $v \leftarrow \mathcal{C}.encode(m) + s \cdot r_2 + e$
- 13: **return** u, v
- 14: **return** $ss, ct = (u, v, salt)$

Algorithm 3 KEM.Decaps (Simplified)

Input: sk, ct
Output: ss

- 1: $seed_{sk}, pk, \sigma \leftarrow sk$
- 2: $\triangleright \text{PKE.Decrypt}(seed_{sk}, ct):$
- 3: $ctx_{sk} \leftarrow \text{XOF.init}(seed_{sk})$
- 4: $x \leftarrow \text{SampleFixedWeight}_s(ctx_{sk}, \omega)$
- 5: $y \leftarrow \text{SampleFixedWeight}_s(ctx_{sk}, \omega)$
- 6: $m \leftarrow \mathcal{C}.decode(v - u \cdot y)$
- 7: **return** m
- 8: $(ss, \theta) \leftarrow \mathcal{G}(\mathcal{H}(pk) || m || salt)$
- 9: $ct' \leftarrow \text{PKE.Encrypt}(pk, m, \theta)$
- 10: **if** $m = \perp \vee ct \neq ct'$:
- 11: $ss \leftarrow \mathcal{J}(\mathcal{H}(pk) || \sigma || ct)$
- 12: **return** ss

Definition 1 (Syndrome Decoding Problem ($SDP(n, k, t)$)).

Let n, k, ω be positive integers. Given $H \in \mathbb{F}_2^{(n-k) \times n}$ and $s \in \mathbb{F}_2^{(n-k)}$, the $SDP(n, k, t)$ asks to find $e \in \mathbb{F}_2^n$ such that $HW(e) = t$ and $He^T = s$.

Specifically for HQC, $H = [I_N, \text{rot}(h)]$, and $e = [x, y]$ where x and y are two secret key vectors, each with exact $HW = t/2$. $\text{rot}(h)$ is the circulant matrix computed from h . The $SDP(n, k, t)$ parameters for all three HQC instances are shown in Table 1.

In general, there is no efficient way to solve SDP. Therefore, HQC ensures its security by carefully choosing the parameters so that it is computationally infeasible to solve its SDP. However, knowing partial information about the vector e can reduce the computational cost to be feasible. For instance, [18] demonstrated that knowing slightly over

50% of the positions of zeros from (x, y) in total can result in obtaining a (over-)determined linear system of equations with high probability, which can be solved by simply using Gaussian elimination.

ISD is another approach to solve the SDP. It is a class of generic decoding algorithms first introduced by [33]. In its simplest form, ISD works by first finding an *information set*, which is defined to be a set $\mathcal{I} \subset \{0, \dots, n-1\}$ of l positions such that the parity check matrix H , with columns in $\mathcal{I}$ removed, is invertible. If the information set does not contain any positions with non-zero error, we may directly solve for the secret key. Many improvements have since been described, e.g., by Lee and Brickel [40] and Stern [41], and more recently in [42], [43]. In general, the running time of this algorithm is determined by finding such an information set in a $SDP(n, k, t)$, which happens with probability [44]:

$$Pr_{succ} = \frac{\binom{n-k}{t}}{\binom{n}{t}} \quad (2)$$

Therefore, the cost of ISD can be estimated as $1/Pr_{succ}$ operations. Under the parameters of HQC, the cost of ISD is $2^{132.35}$, $2^{200.40}$, and $2^{262.43}$, respectively.

ISD has also been used in the context of side-channel attacks, e.g., in [25], [31], [45], [46], [47], [48]. The authors of [49] propose a framework for side-channel attacks against code-based schemes based on ISD. They define so-called ‘hints’, i.e., types of side information, that reveal partial information about the SDP to reduce its hardness. In our work, we will rely on their framework and prove some properties that are specific to HQC and the leakage we obtain. In particular, our work relies on the two following theorems from [49]:

Theorem 1 (Theorem 1 of [49]). *Consider an $SDP(n, k, t)$ defined in Definition 1 with $He^T = s$. If an error location of e is known (i.e., an index j such that $e_j = 1$), then the original $SDP(n, k, t)$ can be transformed into an $SDP(n-1, k-1, t-1)$ or $SDP(n-1, k, t-1)$.*

Theorem 2 (Theorem 3 of [49]). *Consider an $SDP(n, k, t)$ defined in Definition 1 with $He^T = s$. If an error-free location of e is known (i.e., an index j such that $e_j = 0$), then the original $SDP(n, k, t)$ can be transformed into an $SDP(n-1, k-1, t)$ or $SDP(n-1, k, t)$.*

For both the Theorem 1 and Theorem 2, the former case happens if the shortened code $H_{[n] \setminus [j]}(e_{[n] \setminus [j]})^T = s - H_j e_j^T$ where $HW(e_{[n] \setminus [j]}) = t$ has dimension $k-1$, which happens with a high probability for a random code with

large n . Otherwise, the shortened code has dimension k , which leads to the latter case.

2.2. Side-Channel Attacks on HQC

Existing side-channel attacks against HQC targeting the long-term secret key y [16], [17], [18], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31] exploited the known fact that the PKE.Decrypt is vulnerable to a CCA. Therefore, side-channel leakages are used to reveal information about the PKE.Decrypt results that are useful for CCAs. In contrast, our attacks do not depend on the CCA but exploit leaky operations on the secret key.

Among existing attacks, three attacks targeted the timing leakages [16], [17], [18]. The timing leakages exploited by them exist at the source code level, and the attacks required thousands of invalid ciphertexts. Specifically, they all exploited the leaky rejection sampling during the re-encryption to build a so-called binary Plaintext-Checking (PC) Oracle to distinguish whether a chosen-ciphertext is decrypted into an expected message or not. [16] achieved it by using a secret-dependent branch; [50] achieved it by using a secret-dependent memory access; and [18] achieved it by using a secret-dependent integer division. Different from their work, our attacks target HQC implementations that are constant-time at the source-code level but not in the binary. Additionally, our attacks work with an arbitrary ciphertext, which can be a valid ciphertext generated by the victim.

Compared with timing side-channel attacks, power side-channel attacks on HQC [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31] are more common. With power side-channel, previous works achieved a Multi-Value PC Oracle [29], a Full-Decryption PC Oracle [29], [31], or even directly revealed information about a specific intermediate state of the PKE.Decrypt [22], [26], [27], [28], [30], [31]. The state-of-the-art power side-channels have achieved single-trace attacks [22], [27], [28], [31], and even used valid ciphertext only [27], [31], by applying a soft-information analysis [51]. However, all of them are still CCAs and require an active query to the victim device. Notably, the most recent power side-channel attack on HQC [31] positions their attack as a passive attack. However, their threat model still requires an attacker to know the internal randomness generated during the encapsulation $((r_1, r_2, e)$ in Algorithm 2) for a certain ciphertext. It is only possible when the attacker generates the ciphertext by himself. Therefore, their attack is essentially a CCA and is not entirely passive. To the best of our knowledge, our attacks are the first passive attacks on HQC.

2.3. AMD Secure Encrypted Virtualization

AMD Secure Encrypted Virtualization (SEV) [52] is a hardware feature designed to protect the confidentiality of guest Virtual Machines (VMs) from privileged adversaries, such as a compromised or malicious hypervisor. SEV enables per-VM memory encryption, where data is automati-

cally encrypted as it leaves the CPU package for DRAM and decrypted when it is fetched back. AMD evolved this technology with SEV-ES, to protect the guest register state, and SEV-SNP, to provide strong memory integrity and rollback protection via a hardware-managed Reverse Map Table [53].

However, despite these advancements in protecting direct memory access, SEV-SNP VMs remain vulnerable to side-channel attacks [53]. Previous work demonstrates that attackers can exploit such leakage to steal traditional cryptographic keys [54], [55], [56], machine learning model inputs [57], and break privacy guarantees [34]. To collect these side-channel leakages, SEV-STEP [58] provides an interrupt-based framework that enables single-stepping on SEV-SNP VMs. FARFETCH'D [34] uses a similar technique for stepping and employs performance counters to filter tracking. This filtering mechanism accelerates the collection process by skipping irrelevant traces, allowing FARFETCH'D to gather side-channel traces related to guest VM events.

3. Threat Model and Experiment Setup

Our timing side-channel attacks target a victim process that runs HQC decapsulation within a trusted execution environment (TEE). We assume a typical cloud deployment where the victim runs inside an AMD SEV-SNP VM. Following the threat model of AMD SEV [53], the host operating system (hypervisor) is malicious. Hence, an attacker can circumvent noise (e.g., by scheduling the VM to a specific core or disabling hyperthreading) and read Performance Counters (PMCs) for guest (e.g., victim) events. However, the secret key, which is generated by the victim process at runtime, remains confidential as the SEV hardware encrypts and protects it. This threat model aligns with previous attacks that target TEEs [59], [60], [61].

We conduct our attacks on a 16-core AMD EPYC 9124 CPU. The host system is running Ubuntu 24.04 LTS with Linux Kernel 6.11.0. The victim SEV-SNP VM is running Ubuntu 22.04 LTS with Linux Kernel 6.10.0. We target the latest stable HQC implementation provided by PQClean [19]. It is based on the Round 4 reference implementation and is patched with a mitigation for a recent timing attack [18]. The source code is constant-time and thus mitigates all known timing attacks on HQC [16], [17], [18]. An end-user can compile the code with any compiler and optimization level. Here, we assume the victim may either use GCC-13 or Clang-18 together with a common optimization level: `-O0`, `-O1`, `-O2`, `-O3` or `-Os`.

4. Attack 1: Sparse Vector HW Attack

In this section, we present our first attack, which we refer to as *Sparse Vector Hamming Weight Attack*. It leaks the HW of each secret key word when the decapsulation process is re-sampling the secret key (x, y) (Lines 4 and 5 in Algorithm 3). By knowing all key words that have HW of zero, an attacker can recover the remaining bits with Gaussian Elimination. Since re-sampling the key does

not depend on the received ciphertext, our attack is thus ciphertext-independent. We first explain the leakage we find and then illustrate how to exploit it to recover the full secret key. Finally, we demonstrate our attack and explain how we solve practical challenges.

4.1. Compiler-induced Side-Channel Leakage

Recall that HQC does not store the long-term secret key in memory. Instead, it retains a seed_{sk} from which the key can be deterministically reconstructed. During each decapsulation, HQC unpacks the seed_{sk} and re-generates the key (x, y) by inserting 1s into two sparse vectors. To prevent leakage of the modified word positions or the number of 1s written, HQC adopts the *Constant Weight Word Sampling* algorithm proposed by Sendrier [62].

Constant-time Source Code. Listing 1 presents a simplified version of key re-sampling, where we only show code relevant to our attack. The full version is available on GitHub (see GitHub Link). Parameter ctx is the state of the pseudo-randomness, which is derived from a seed , and weight specifies the number of 1s that should be written to a sparse vector. The sampled vector, either x or y , is stored in v , an array of words. By default, v consists of zeros only.

The function first samples which word of v should be written 1s and stores the corresponding indices in array index_tab . If a word inserts two 1s, it takes two entries in the array. For each index_tab entry, it samples the bit position that should be set to 1 and stores the bit mask in array bit_tab . The Hamming Weight of each bit_tab entry is exactly one. After initializing the two arrays, it writes 1s to v with a classic constant-time bitmasking technique (Lines 9 to 13). Specifically, it iterates through each word of vector v (i.e., $v[i]$), always reads from each index_tab and bit_tab (i.e., $\text{index_tab}[j]$ and $\text{bit_tab}[j]$), and always writes to the vector. It only writes the value of $\text{bit_tab}[j]$ to $v[i]$ when the index of the iterated word (i) matches the value of $\text{index_tab}[j]$. Otherwise, it writes zero to the vector. Therefore, the number of executed memory accesses do not depend on the value being written to the key.

Leaky Binary. We compile the code with Clang-18 and $-\text{O}1$, and obtain a variable-time binary. It leaks the value of tmp through a conditional branch. We present the related assembly code of the leaky binary in Listing 2. The full assembly is available in the Godbolt online compiler (Godbolt Link). In general, the assembly code follows the structure of the source code. The only exception is that it introduces an additional conditional branch at assembly code block `.LBB0_13` to check if weight is zero. If so, it skips the entire iteration of $\text{index_tab}[j]$, which we refer to as the inner loop. Since weight is never zero, this branch is never taken. The code block `.LBB0_15` iterates over $v[i]$, which we refer to as the outer loop. Different from the source code, it first writes val to $v[i]$ and then moves to the next word of v . Similarly, the assembly code first writes to val and then moves to the next $\text{index_tab}[j]$ at code block `.LBB0_21`.

The inner loop body is presented in code block `.LBB0_18`. Here, the compiler realizes that the sequence of

Listing 1 Fixed-hamming-weight binary vector Sampling

```
1 void vect_set_random_fixed_weight
2 (state *ctx, uint64_t *v, uint16_t weight){
3     // sample bit_tab[] index_tab[] from ctx
4     ...
5     // Writing 1s to the vector
6     for (size_t i = 0; i < VEC_N_SIZE_64; ++i){
7         val = 0;
8         for (size_t j = 0; j < weight; ++j){
9             tmp = (uint32_t)(i - index_tab[j]);
10            tmp = 1 ^ ((uint32_t)
11                (tmp | (0 - tmp)) >> 31);
12            mask64 = 0 - (uint64_t)tmp;
13            val |= (bit_tab[j] & mask64);
14        }
15        v[i] |= val;
16    }
17 }
```

bitwise operations highlighted in Listing 1 is redundant and replaces them with a conditional branch. Specifically, the comparison at Line 23 loads $\text{index_tab}[j]$ and checks if it is equal to i . If so, it branches to code block `.LBB0_20` to write $\text{bit_tab}[j]$ to val . Otherwise, it skips the write and directly branches to the iteration of $\text{index_tab}[j]$ in block `.LBB0_21`. Obviously, the assembly code is not constant-time as taking a conditional branch and writing to memory or not depend on the value being written to the secret key. If the conditional branch at Line 24 is never taken during one entire inner loop iteration, the corresponding $v[i]$ would be zero. Otherwise, the number of times this branch is taken indicates how many 1s are written to $v[i]$. Therefore, monitoring the conditional branch at Line 24 leaks the Hamming Weight of every 64 bits of the secret key.

4.2. Full Key Recovery Strategy

Knowing more than 50 % bits of (x, y) leads to a high chance that one can obtain a unique solution by solving the public key relation $s = x + h \cdot y$ with Gaussian Elimination. As shown by Schröder et al. [18], the success rate when an attacker knows 50 % + 5 bits is 97 %.

We take HQC-128 as an example to explain how the same cryptanalysis applies to our attack. In HQC-128, each key vector has 17 669 bits and only 66 of them are 1s. The 17 669 bits are grouped into 277 words of 64-bit length. At least 211 of them would be zero, assuming the worst case where the maximum HW of each word is one. Therefore, knowing the positions of all zero blocks already recovers 76 % of the key. According to [18], the success rate of obtaining a unique solution from Gaussian Elimination is 100 %.

It is even easier to recover the key for HQC with higher security levels. For HQC-192, each secret key vector has 35 851 bits, and only 100 of them are 1s. When dividing the vector into 561 64-bit words, at least 461 blocks would be zero. Therefore, the side-channel attack directly leaks 82 % of the bits in each vector. For HQC-256, only 131 bits out

Listing 2 Leaky assembly of writing 1s to secret key.

```

1  ...
2  .LBB0_14:
3  xorl %ecx, %ecx
4  .LBB0_15:
5  orq  %rcx, (%rdi,%rax,8) ; v[i] != val
6  incq %rax ; Iterating i in VEC_N_SIZE_64
7  cmpq $277, %rax
8  je   .LBB0_16 ; Complete Writing to v
9  .LBB0_13:
10 testw %r14w, %r14w ; Is weight zero?
11 je   .LBB0_14 ; Skip the inner loop if so
12 xorl %edx, %edx
13 xorl %ecx, %ecx
14 jmp  .LBB0_18
15 .LBB0_20: ; write bit_tab[j] to val
16 movq 944(%rsp,%rdx,8), %rsi ; <-- Leak
17 .LBB0_21: ; Iterating j until weight
18 orq  %rsi, %rcx
19 incq %rdx
20 cmpq %rdx, %r12
21 je   .LBB0_15
22 .LBB0_18: ; Check if i == index_tab[j]
23 cmpl %eax, 336(%rsp,%rdx,4)
24 je   .LBB0_20 ; <-- Leak
25 xorl %esi, %esi
26 jmp  .LBB0_21
27 ...

```

of 57637 bits are 1s. Therefore, at least 770 words out of 901 words would be zero, which means that a side-channel attack directly leaks 86 % key bits.

4.3. End-to-end Side-Channel Attack

We conduct an end-to-end side-channel attack to confirm that an attacker can practically extract all required side-channel information by ❶ generating a random key and ciphertext, ❷ passively monitoring victim behavior with FARFETCH'D, and ❸ automatically extracting leakages.

Monitoring Victim Behavior. We monitor the execution of HQC decapsulation that is protected by AMD SEV-SNP virtual machine with the FARFETCH'D framework [34]. The framework single-steps the victim process and leaks per-instruction information with Performance Monitor Counters (PMCs). We configure it to monitor three PMCs [63]: *Retired Instructions*, *Retired Branch Instructions*, and *Retired Taken Branch Instructions*. The first PMC tells us how many instructions are executed between each single-step (ideally, one). The second PMC indicates whether the single-stepped instruction is a branch, while the last PMC reports whether a branch is taken. Following the method proposed by Gast et al. [64], we reconstruct the control flow of the victim process executing inside the protected virtual machine by combining these counters.

Profiling Victim Behavior. Although FARFETCH'D single-steps an SEV-SNP virtual machine, it does not leak the guest program counter. Therefore, an attacker must profile the victim's behavior to locate the PMC results related to the secret. This process is not trivial and often requires

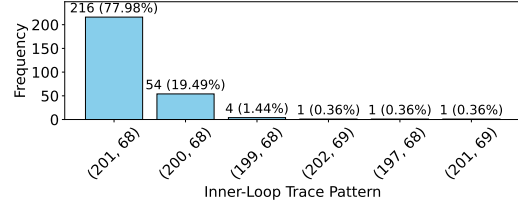


Figure 1. PMC pattern frequency of inserting 1s to a sparse vector.

significant human effort. However, we find that an attacker can automatically detect the leaky instructions, highlighted in Listing 1, by searching for PMC traces correlated to a specific execution pattern. With this pattern, an attacker can automatically identify each iteration of $v[i]$, and thus infer the total number of taken branches in the inner loop.

This specific pattern occurs when ❶ leaves the inner loop, ❷ moves to the next $v[i]$, and ❸ re-enters the inner loop. Specifically, execution takes the conditional branch at Line 21 in Listing 2 to leave the inner loop and, after three retired instructions, executes another branch at Line 8. This branch is not taken to keep iterating $v[i]$. After one more retired instruction, execution reaches the branch at Line 11 (which checks if weight is equal to zero), but execution never takes it. Finally, after two more retired instructions, execution takes a direct branch at Line 14 to execute the body of the inner loop. To summarize, the pattern we use to separate each iteration of the outer loop is: TakenBr $\rightarrow$ 3 RetiredInstrs $\rightarrow$ NotTakenBr $\rightarrow$ 1 RetiredInstr $\rightarrow$ NotTakenBr $\rightarrow$ 2 RetiredInstrs $\rightarrow$ TakenBr.

This execution pattern is deterministic and is guaranteed to appear $VEC_N_SIZE_64 - 1$ times as the function returns after the last iteration of the outer loop. Hence, for HQC-128, where $VEC_N_SIZE_64$ is 277, this pattern always appears 276 times when sampling a key vector.

Leaking Secret-Dependent Branch Outcome. After identifying all 277 executions of the inner loop, we now recover their HWs. We focus on finding the PMC trace pattern that corresponds to *never* taking the conditional branch when iterating all $index_tab$, as knowing all zero words is sufficient to recover the key. Since most words are zero, the trace pattern of interest is the one that occurs most frequently. We find this pattern with frequency analysis and consider the one that has the highest frequency to be the pattern of interest. More concretely, we count the pair of *Retired Branches* and *Retired Taken Branches* for each trace block. An example result is presented in Figure 1. As expected, there is only one pattern that occurs about 78 % of the time.

Practical Success Rate. We conduct 100 independent attacks, 10 attacks for each randomly generated secret key. Thanks to the stability of FARFETCH'D, our analysis script always *automatically* identifies each iteration of $v[i]$ and identifies the PMC pattern of processing $v[i]$ of Hamming Weight zero. Therefore, the practical success rate is 100 %.

Problematic Compiler Optimization Investigation. We also investigate the possibility of carrying out the attack on binaries obtained with different compilers and compilation

optimizations. We find that GCC-13 does not generate vulnerable binaries under any optimization level. On the other hand, Clang-18 generates variable-time binary under all optimization levels except -O0. According to our evaluation, an attacker can practically extract precise and sufficient leakages from binaries obtained with -O1 and -Os. However, both optimization levels of -O2 and -O3 vectorize the binary and thus the secret-dependent branch is not taken in most scenarios. Consequently, the PMC pattern that occurs the most does not necessarily correspond to processing the word with HW of zero.

5. Extending Attack 1 to Power Side-Channel

The information we obtain from the timing side-channel attack may also be extracted from a power side-channel that targets embedded devices. In general, power side-channel attacks cannot extract noise-free HW of the targeted variable from a properly implemented cryptographic code. Instead, they obtain a noisy version or even just the Hamming Distance. However, a ‘careless’ implementation may amplify the signal and allow for the extraction of single bits or even noise-free HWs. For example, the work of [65] shows that a recently proposed protected gadget for ML-KEM leaks single bits with high probability; the authors of [66] provide a more general analysis of this phenomenon in the context of protected implementations of post-quantum schemes.

The sparse vector HW leakage we found in HQC is another case where software implementation amplifies power side-channel attacks. In Listing 1, `mask64` is an unsigned 64-bit variable. It contains all zeros when `bit_tab[j]` should be discarded or all ones when `bit_tab[j]` is kept. Flipping all bits of `mask64`, from zero to one or vice versa, causes significant power changes. By monitoring the power change of updating this variable, an attacker can recover the HW of each sparse vector word. Therefore, a power side-channel attacker can also recover the secret key of HQC with arbitrary ciphertext, which removes the chosen-ciphertext condition required by previous power attacks [20], [21], [22], [23], [24], [25], [26], [27], [28], [29], [30], [31].

Proof-of-Concept Experiment. We show that it is practical to extract noise-free HWs of each key word by measuring the power changes of `mask64` at Line 12 in Listing 1. We conduct a Proof-of-Concept (PoC) experiment on a ChipWhisperer-Lite-ARM board equipped with an STM32F303RCT7 target processor. The target processor runs at a clock frequency of 7.37 MHz. The power traces were collected using a 10-bit Analog-to-Digital Converter operating at the same clock frequency as the target processor. During the experiment, we first compute the tables of `bit_tab` and `index_tab` and then monitor the power traces of processing each vector word.

Figure 2 presents the power traces for processing `v[i]` with three HWs: the top one for HW of zero, the middle one for HW of one, and the bottom one for HW of two. They are the results of single-trace measurements taken individually. Between the sampling points of 250 and 1800 are the results related to one full iteration of writing 1s to `v[i]`. As shown

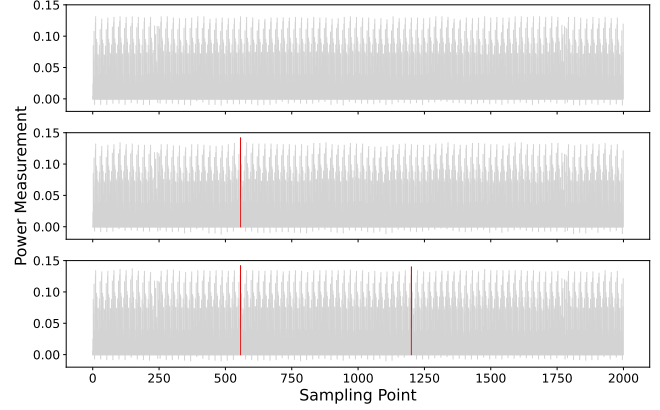


Figure 2. Single-trace measurement results for `v[i]` with HW of zero, one, and two, respectively.

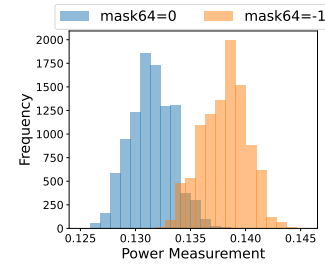


Figure 3. The histogram distribution of power measurements at the point when `mask64` holding all zeros and holding all ones.

in the figure, the power measurement patterns around the sampling points at 250 and 1800 visually differ from those at the other positions. They correspond to operations that finish processing one vector word and move to the next word. We highlight the power measurements that indicate `mask64` contains all 1s. Statistically, their power measurements are higher than the rest. It is more obvious from Figure 3, where we perform 10 000 measurements and present the power measurement distribution at the point when `mask64` holds all zeros or all ones. Despite the small overlap between the distributions, they can still be clearly distinguished.

Discussion. Our results show that the HW of a word can be leaked when re-sampling the key. In that case, we get direct leakages on the sparse secret to enable a purely passive attack, which is opposed to CCAs. The PQClean implementation we target is not explicitly tailored for embedded devices, while real-world implementations may additionally include countermeasures. Nevertheless, it has already been targeted in several previous side-channel attacks [24], [29], [30]. Furthermore, as shown by previous work [65], [66], implementations protected by countermeasures may also suffer from similar vulnerabilities. Our PoC demonstrates that our *Sparse Vector Hamming Weight Attack* on key re-sampling is not limited to timing side-channels but can also be extended to power side-channels. Therefore, these sparse vector operations need careful protection when deploying

those implementations on embedded devices.

6. Generalizing Attack 1 to Code-based KEM

The sparse-vector Hamming Weight leakage not only exists in the current PQClean HQC implementation, but also remains in the latest version released after our initial responsible disclosure. Furthermore, it also exists in all the other code-based KEMs in the Round 4 NIST PQC submission, specifically, in BIKE and Classic McEliece. This finding shows that implementing constant-time code-based KEM is generally challenging. In this section, we first show that one can use Information-Set Decoding (ISD) to recover the long-term secret key for the latest release of HQC. Then, we explain how to recover the session key of BIKE and McEliece based on side-channel leakages and present two PoC attacks.

6.1. New HQC Implementation (Aug 22 2025)

After our initial responsible disclosure, the HQC team releases a new implementation on Aug 22, 2025 (Official Repository Link, with algorithms in Appendix A), which is claimed to be constant-time [4]. It modifies the secret key sampling process such that only $\mathbf{y}$ is re-sampled in the KEM.Decaps. This change is reasonable as only $\mathbf{y}$ is used to decrypt the ciphertext. However, the vulnerable function shown in Listing 1 is still used to write 1s to $\mathbf{y}$. Therefore, an attacker can still reveal the HW of each word of $\mathbf{y}$ with the same attack technique presented in Section 4.

Without information about $\mathbf{x}$, we can no longer recover the remaining bits with Gaussian Elimination, as we know less than 50% information about the secret key $(\mathbf{x}, \mathbf{y})$ this time. However, we can still recover the remaining bits of $\mathbf{y}$ with ISD. Recall that at least 211 out of 277 words of the HQC-128 secret key are zero. Knowing the positions of these words is equivalent to knowing about 13504 bits (76%) of $\mathbf{y}$. According to [49], see Theorem 2, knowing an error-free position allows transforming the $\mathcal{SDP}(n, k, t)$ into an $\mathcal{SDP}(n-1, k-1, t)$ or $\mathcal{SDP}(n-1, k, t)$, where we are in the former case with high probability.

In the case where we only obtain information on $\mathbf{y}$ and $\mathbf{h} \in \mathcal{R} = \mathbb{F}_2[x]/(x^N - 1)$ with primitive N (which is the case in HQC), we may precisely compute probabilities that the former case achieved. Recall that we are in the former case if the columns in $\mathbf{H} = [\mathbf{I}_N, \text{rot}(\mathbf{h})]$ at the positions that correspond to the recovered coefficients in $\mathbf{e}$ are independent. As we only obtain information on $\mathbf{y}$, all our columns are chosen from $\text{rot}(\mathbf{h})$. Therefore, to compute the probability of obtaining dependent columns, it suffices to compute the rank of $\text{rot}(\mathbf{h})$ for all $\mathbf{h}$. In HQC, N is primitive and therefore, $x^N - 1 = (x-1)(x^{N-1} + x^{N-2} + \dots + 1) = (x-1)\phi(x)$ and by the Chinese Remainder Theorem (CRT), we have

$$\mathcal{R} \cong \mathbb{F}_2[x]/(x-1) \times \mathbb{F}_2[x]/(\phi(x)) \cong \mathbb{F}_2 \times \mathbb{F}_2^{N-1} \quad (3)$$

For $\mathbf{h} = (a, b) \in \mathbb{F}_2 \times \mathbb{F}_2^{N-1}$ with **1** $a, b \neq 0$, $\text{rot}(\mathbf{h})$ is invertible (and thus of full rank). If $\mathbf{h}$ is of the form **2**

$(0, b)$ with $b \neq 0$, the kernel of multiplication with $\mathbf{h}$ is $(a, 0)$ for $a \in \mathbb{F}_2$, which is a 1-dimensional subspace. Thus, the rank of $\text{rot}(\mathbf{h})$ is $N-1$. For $\mathbf{h}$ being of the form **3** $(a, 0)$ with $a \neq 0$, the kernel is $(0, b)$, i.e., a $N-1$ dimensional subspace and thus the rank of $\text{rot}(\mathbf{h})$ is 1. If **4** $\mathbf{h} = 0$, the associated matrix is, of course, the zero matrix. In total, we have $2^{N-1} - 1$ elements for which $\text{rot}(\mathbf{h})$ is of full rank N , $2^{N-1} - 1$ elements for which $\text{rot}(\mathbf{h})$ is of rank $N-1$, 1 element for which $\text{rot}(\mathbf{h})$ is of rank 1, and one element for which it is of rank 0. The last two cases ($(a, 0)$ with $a \neq 0$ and $\mathbf{h} = 0$) happen with probability $1/2^{N-1}$ each and are therefore irrelevant in practice (the smallest N in HQC is 17669).

Assume we recover l error-free positions from $\mathbf{y}$: We have roughly 50% probability that $\text{rot}(\mathbf{h})$ has full rank (case **1**). In that case, we always reduce the $\mathcal{SDP}(2N, N, 2\omega)$ into $\mathcal{SDP}(2N-l, N-l, 2\omega)$. With roughly 50% probability (case **2**), $\text{rot}(\mathbf{h})$ has rank $N-1$, and reduce the SDP to either an $\mathcal{SDP}(2N-l, N-l, 2\omega)$ or an $\mathcal{SDP}(2N-l, N-l+1, 2\omega)$. In the latter case, the recovered error-free positions contain the one dependency introduced by the rank drop in $\text{rot}(\mathbf{h})$. Therefore, knowing 76% bits of $\mathbf{y}$ in HQC-128, which is already the worst case, reduces the cost of solving the $\mathcal{SDP}$ to around $2^{40.44}$. This cost would be around $2^{47.15}$ for HQC-192 and $2^{51.29}$ for HQC-256. Notably, all of them are below the security strength that allows a brute-force enumeration before year 2031 by NIST [67], which is 2^{112} , and the security strength set by ANSSI [68], which is 2^{70} . Thereby, solving for the secret key is costly but feasible with currently available computing resources.

6.2. BIKE

BIKE does not re-sample the secret key in its decapsulation process, but still re-generates the error vectors (e'_0, e'_1) from the decrypted message m' during the decapsulation. Leaking the HW of each error vector word allows an attacker to recover the decrypted message and get the shared session key. We analyze its latest official implementation (Link), which claims to be constant-time (Link), and find that its binary does leak the HW when compiled with GCC-13-00. We first explain the leakage, then introduce the key reconstruction strategy, and finally present a PoC attack.

Leaky Binary. BIKE writes 1s to the error vector in a similar way to HQC (recall Listing 1). Specifically, it also uses a nested loop to iterate w times for each word of an error vector (Code Snippet). The source code is constant-time as it uses a 64-bit mask to conditionally write 1s to a word. The comparison used to update the mask is implemented as a function call, which is also constant-time. When compiling with -O0, this function call is not inlined. Instead, it uses a conditional branch to update the mask based on the function call's return (Godbolt Example). When the function returns one, the code takes a conditional branch to set the mask to be all ones. Otherwise, it does not take the branch and sets the variable to zero. Therefore, leaking the number of times the conditional branch is taken leaks the HW of a word.

Algorithm 4 BIKE Encaps (Simplified)

Input: $\text{pk} = \mathbf{h}$
Output: $\text{ss}, \text{ct} = (\mathbf{c}_0, \mathbf{c}_1)$

- 1: $\mathbf{m} \leftarrow \text{Random}()$
- 2: $(\mathbf{e}_0, \mathbf{e}_1) \leftarrow \mathcal{H}_t(\mathbf{m}, \mathbf{h}[:l_{bits}])$
- 3: $\mathbf{c}_0 \leftarrow \mathbf{e}_0 + \mathbf{e}_1 \cdot \mathbf{h}$
- 4: $\mathbf{c}_1 \leftarrow \mathbf{m} \oplus \mathcal{L}(\mathbf{e}_0, \mathbf{e}_1)$
- 5: $\text{ss} = \mathcal{K}(\mathbf{m}, \mathbf{c}_0, \mathbf{c}_1)$
- 6: **return** $\text{ss}, \text{ct} = (\mathbf{c}_0, \mathbf{c}_1)$

Algorithm 5 BIKE Decaps (Simplified)

Input: $\text{sk} = (\mathbf{h}_0, \mathbf{h}_1, \mathbf{h}[:l_{bits}], \sigma), \text{ct} = (\mathbf{c}_0, \mathbf{c}_1)$
Output: ss

- 1: $(\mathbf{e}_0, \mathbf{e}_1) \leftarrow \text{decoder}(\mathbf{c}_0 \cdot \mathbf{h}_0, \mathbf{h}_0, \mathbf{h}_1)$
- 2: $\mathbf{m}' \leftarrow \mathbf{c}_1 \oplus \mathcal{L}(\mathbf{e}_0, \mathbf{e}_1)$
- 3: $(\mathbf{e}'_0, \mathbf{e}'_1) \leftarrow \mathcal{H}_t(\mathbf{m}', \mathbf{h}[:l_{bits}])$
- 4: **if** $(\mathbf{e}_0, \mathbf{e}_1) \neq (\mathbf{e}'_0, \mathbf{e}'_1)$:
- 5: **return** $\text{ss} \leftarrow \mathcal{K}(\sigma, \mathbf{c}_0, \mathbf{c}_1)$
- 6: **return** $\text{ss} \leftarrow \mathcal{K}(\mathbf{m}', \mathbf{c}_0, \mathbf{c}_1)$

Key Reconstruction Strategy. We provide the algorithms of BIKE Encapsulation and Decapsulation in Algorithm 4 and Algorithm 5. $\mathcal{H}_t$ expands the input as a seed to generate error vectors while both $\mathcal{L}$ and $\mathcal{K}$ are hash functions. Recovering the session key requires an attacker to know the ciphertext being decapsulated. It can be any valid ciphertext $\text{ct} = (\mathbf{c}_0, \mathbf{c}_1)$ generated by the victim. Therefore, the decrypted message $\mathbf{m}'$ is the same as $\mathbf{m}$, which is randomly generated during the encapsulation. Similarly, the $(\mathbf{e}'_0, \mathbf{e}'_1)$ re-generated with $\mathbf{m}'$ and public key $\mathbf{h}$ would be the same as $(\mathbf{e}_0, \mathbf{e}_1)$ generated during the encapsulation. Thus, the known ciphertext relation $\mathbf{c}_0 = \mathbf{e}_0 + \mathbf{e}_1 \cdot \mathbf{h}$ (Line 3 in Algorithm 4) is also satisfied by $\mathbf{c}_0 = \mathbf{e}'_0 + \mathbf{e}'_1 \cdot \mathbf{h}$. Recall that the side-channel attack leaks the HW of each word of $\mathbf{e}'_0$ and $\mathbf{e}'_1$. In BIKE-128, both $\mathbf{e}'_0$ and $\mathbf{e}'_1$ have 12 323 bits, which are divided into 386 words in total, and the sum of their HW is 134. Therefore, assuming the worst case that the HW of each word can at most be one, an attacker can at least recover 65 % of the bits being zeros. With Gaussian Elimination, the remaining bits of two error vectors can be recovered with the known ciphertext relation. With the recovered $(\mathbf{e}'_0, \mathbf{e}'_1)$ and known ciphertext $\mathbf{c}_1$, we can compute $\mathbf{m}' \leftarrow \mathbf{c}_1 \oplus \mathcal{L}(\mathbf{e}'_0, \mathbf{e}'_1)$. The session key is thus obtained with $\text{ss} \leftarrow \mathcal{K}(\mathbf{m}', \mathbf{c}_0, \mathbf{c}_1)$.

PoC Attack. We conduct a PoC attack, which is very similar to the Attack 1 presented in Section 4. The only difference is the PMC patterns of writing 1s to each word. However, the pattern of HW zero can be automatically identified with the same frequency analysis methodology. We confirm that an attacker can learn precise information about the number of taken branches in the nested loop.

6.3. Classic McEliece

Different from BIKE, Classic McEliece only generates an error vector in the encapsulation process, but not in the decapsulation process. The encapsulation generates both the session key, which is kept secret, and the ciphertext, which

Algorithm 6 Classic McEliece Encaps (Simplified)

Input: $\text{pk} = \mathbf{T}$
Output: ss, ct

- 1: $H \leftarrow (\mathbf{I}|\mathbf{T})$
- 2: $\mathbf{e} \leftarrow \text{SampleFixedWeight}(t)$
- 3: $\text{ct} \leftarrow \mathbf{H}\mathbf{e}$
- 4: $\text{ss} \leftarrow \mathcal{H}(1, \mathbf{e}, \text{ct})$
- 5: **return** ss, ct

is made public. Therefore, an attacker aims to recover the session key with the known ciphertext and side-channel leakages during the encapsulation. We analyze the latest PQClean implementation of Classic McEliece (Link), whose source code is constant-time, and find that its binary does leak the HW of each byte of the error vector when compiled with Clang-18 -O1. Consequently, an attacker can passively monitor the encapsulation process to leak bits about the error vector and thus recover the session key.

Leaky Binary. Classic McEliece writes 1s to the error vector in a similar way to HQC and BIKE. The difference is that it groups the error vector into bytes instead of words. Therefore, it uses an 8-bit mask to conditionally write error bits into each byte of the error vector. With compiler optimizations, the bitwise operations are compiled to a conditional branch, which is taken when the error bits need to be rewritten to the vector. The source code of the vulnerable code snippet is available at GitHub Link and the leaky assembly code is available at Godbolt Example.

Key Reconstruction Strategy. Algorithm 6 shows the key encapsulation algorithm of Classic McEliece. The sparse vector $\mathbf{e}$, which is the error vector, is sampled from *SampleFixedWeight* with Hamming weight of t . In McEliece-348864, $\mathbf{e}$ is 3488 bits long with a Hamming weight of 64. Therefore, it has 436 bytes and, in the worst case, 372 of them would be zero. Knowing the position of all zero bytes recovers at least 85 % bits of the error vector. The remaining bits can be recovered using Gaussian Elimination to solve the ciphertext equation $\text{ct} = \mathbf{H}\mathbf{e}$ (Line 3 of Algorithm 6). Finally, the attacker can compute the shared session key by $\text{ss} = \mathcal{H}(1, \mathbf{e}, \text{ct})$, where $\mathcal{H}$ is a public hash function.

PoC Attack. We conduct a PoC attack, which is very similar to the Attack 1 presented in Section 4. The differences are that we now passively monitor the encapsulation process, and the PMC pattern associated with HW of zero is changed. However, with frequency analysis, we can still automatically identify all patterns that correspond to processing the HW of zero. We confirm that FARFETCH'D can precisely capture all conditional branch outcomes, which leads to the recovery of all zero bytes of $\mathbf{e}$.

7. Attack 2: Vector Multiplication Attack

Besides the *Sparse Vector Hamming Weight Attack* that applies to all three code-based KEMs in the Round 4 NIST PQC submission, HQC is further vulnerable to *Vector Multiplication Attack*, which is our second attack. Specifically, we show that partial bits of the secret key vector $\mathbf{y}$ can be leaked

Listing 3 Simplified code for Karatsuba Base Case

```

1 void base_mul(uint64_t *c, uint64_t a,
2 uint64_t b) {
3     // Init u[16] from b
4     uint64_t u[16] = .....;
5
6     uint64_t tmp1, tmp2, l, g, h;
7     g = 0;
8     tmp1 = a & 0x0f;
9     // Multiplies 4 LSBs of a with b
10    for (size_t i = 0; i < 16; ++i) {
11        tmp2 = tmp1 - i;
12        g ^= (u[i] & (uint64_t)(0 - (1 -
13            ((uint64_t)(tmp2 | (0 - tmp2)) >> 63)))));
14    }
15    l = g;
16    h = 0;
17    // Multiplies [63: 4] bits of a with b
18    for (size_t i = 4; i < 64; i += 4) {
19        g = 0;
20        tmp1 = (a >> i) & 0x0f;
21        for (size_t j = 0; j < 16; ++j) {
22            tmp2 = tmp1 - j;
23            g ^= (u[j] & (uint64_t)(0 - (1 -
24                ((uint64_t)(tmp2 | (0 - tmp2)) >> 63)))));
25        }
26        l ^= g << i;
27        h ^= g >> (64 - i);
28    }
29    // Repairing 4 MSBs of b
30    ...
31 }

```

via side-channel when multiplying it with the ciphertext $\mathbf{u}$ (i.e., $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$). The leakage only exhibits when enabling compiler optimizations, and it leaks 60 bits from each 64-bit-length key word. Therefore, it is even feasible to brute-force the remaining bits with side-channel information. We first explain the leakage and then explain the key recovery algorithm. Finally, we present our practical attack.

7.1. Compiler-induced Side-Channel Leakage

The latest stable HQC implementation performs large-number multiplication with the Karatsuba Algorithm [69] and uses the *Word-by-word multiplication with repair steps* algorithm [70] for the base case. The base case multiplies two words, $\mathbf{a}$ and $\mathbf{b}$, by splitting $\mathbf{a}$ into 16 4-bit blocks. First, it pre-computes a table that consists of all results of multiplying a 4-bit block with 60 LSBs of $\mathbf{b}$. The 4 MSBs of $\mathbf{b}$ are temporarily ignored to avoid overflow. Then, it performs the multiplication by looking up this table with all 16 4-bit blocks. Finally, it repairs the missing 4 MSBs of $\mathbf{b}$. **Constant-time Source Code.** Listing 3 presents a simplified implementation of the base case multiplication. It only presents the second step, which multiplies all bits of $\mathbf{a}$ with 60 LSBs of $\mathbf{b}$ using 4-bit windows (See full implementation at GitHub Link.). Instead of performing 16 table lookups in a loop, it first looks up the 4 LSBs of $\mathbf{a}$ and then looks up the remaining 15 blocks in a loop (Lines 18 to 28). The table lookup operation accesses all table entries regardless of the

Listing 4 Leaky binary of Karatsuba Base Case

```

1 .LOOPENTRY:      ; Start processing a 4 bits
2     ...
3     JAE LEAVELOOP ; Exit the loop if i == 60
4 .Label1:
5     ...          ; move u[1] to a register
6     cmp RSI, 1   ; rsi stores tmp1
7     JE LABEL2
8     ...          ; clear the register
9 .LABEL2:
10    ...
11    CMP RSI, 2
12    JE LABEL3
13    ...          ; Skipping CMP RSI with 3 ~ 14
14 .LABEL15:
15    ...          ; MOV u[15] to a register
16    CMP RSI, 15
17    JE LOOPENTRY ; Move to the next 4 bits of a
18    ...          ; Clear the register
19    JMP LOOPENTRY

```

value of each 4-bit block. After that, HQC uses the classic bitwise operations to only keep the desired result. Therefore, the source code for multiplying two words is constant-time.

Leaky Binary. However, compiler optimization is smart enough to realize that the bitwise operations can be replaced with a conditional branch. In fact, we find that the optimization made by Clang-18 is more aggressive under `-O2`. It not only replaces the bitwise operations but also unrolls the loop at Line 21 of Listing 3. We present a simplified vulnerable assembly code at Listing 4, where we only present key-dependent conditional branches. The complete assembly code is available at Online Compiler Demo. As we can see from the listing, the loop is unrolled to contain 15 conditional branches (from `.LABEL1` to `.LABEL15`). Each branch checks if `tmp1`, stored in `RSI`, is equal to `1 ~ 15`. If so, it takes the branch and keeps the corresponding table entry, which is loaded before each comparison. Otherwise, it does not take the branch and clears the loaded value. An interesting observation is that the unrolled loop does not check if `tmp1` is equal to 0. It is because the first entry of the pre-computed table is zero, which is known at compile time. Writing zero to `tmp1` serves as a default case when none of the previous 15 branches is taken. Therefore, `tmp1` is zero if none of the branches is taken. Otherwise, its value corresponds to the branch that is taken.

7.2. Key Recovery Strategy

This leaky vector multiplication is invoked when computing $\mathbf{v} - \mathbf{u} \cdot \mathbf{y}$. The secret key $\mathbf{y}$ is passed to the base case of Karatsuba as parameter $\mathbf{a}$. Therefore, an attacker learns 60 bits of each key word through a side-channel attack, and the 4 LSBs of each key word remain unknown. Due to the property of a sparse vector, most of the unknown bits would be zero, and only very few of them would be one. We compute the number of expected remaining bits (denoted as num_{bits}) and remaining ones (denoted as num_{ones}) that

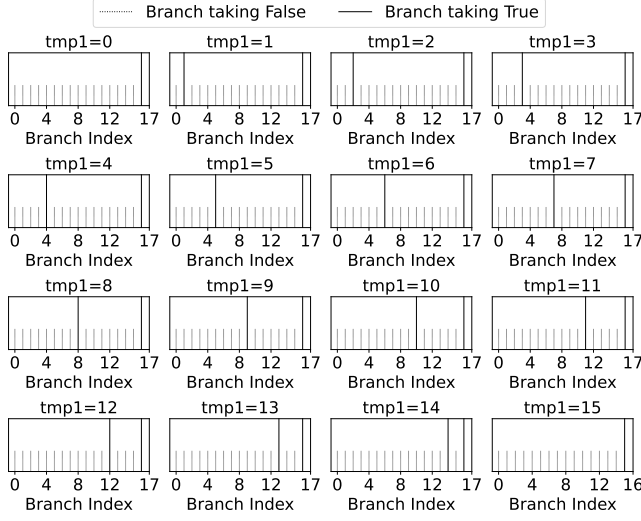


Figure 4. PMC patterns for all possible values of tmp1

cannot be obtained from the side-channel leakages with the following equations:

$$num_{bits} = \lceil \frac{4}{64} \cdot \lceil N/64 \rceil \rceil \quad (4)$$

$$num_{ones} = \lceil \frac{4}{64} \cdot \omega \rceil \quad (5)$$

Therefore, to recover the secret key of HQC-128, an attacker is expected to find four 1s among 1108 bits. For HQC-192, an attacker is expected to find six 1s among 2244 bits. For HQC-256, an attacker is expected to find eight 1s among 3604 bits. With the large amount of recovered bits in y , the attacker can apply ISD to solve the reduced SDP. Here, we not only recovered 0s but also a large amount of 1s. Thereby, the attacker can combine both Theorem 1 and Theorem 2 to reduce the cost of SDP to around $2^{5.82}$, $2^{8.92}$, and $2^{11.77}$ for HQC-128, HQC-192, and HQC-256.

Additionally, the information leaked by the side channel even allows an attacker to recover the key with brute-force enumeration. The total number of possible key candidates, num_{keys} can be computed with the following equation:

$$num_{keys} = \binom{num_{bits}}{num_{ones}} \quad (6)$$

Therefore, the cost to brute-force HQC-128, HQC-192 and HQC-256 are $2^{35.86}$, $2^{57.29}$ and $2^{79.21}$, respectively. For HQC-128, it is totally feasible to brute-force all key candidates on a normal machine. It takes significantly more computational resources to brute-force the remaining bits of HQC-192 and HQC-256.

7.3. End-to-End Side-Channel Attack

Using the FARFETCH'D configuration from Section 4.3, we show that vector multiplication leakage enables an end-

to-end side-channel attack. The challenge is to automatically extract leakages from PMC traces.

Patterns of Leaky Karatsuba Base Case. Karatsuba is a recursive function with complex control flow transfers. We do not find a unique pattern that can automatically separate each invocation of Karatsuba base case and each iteration of processing a 4-bit block of a (i.e., tmp1 at Line 20 in Listing 3). However, we find that the leaky code, presented in Listing 4, only has 16 possible patterns, one for each possible value of tmp1. For example, when the 4-bit block is zero, the pattern will always be: *Not take the branch that leaves the loop* (Line 3 in Listing 4) $\rightarrow$ *Not take any of the 15 key-dependent branches* $\rightarrow$ *Take the backward branch to .LOOPENTRY* (Line 19 in Listing 4). When the value of the block is one, the branch that leaves the loop is not taken, the first of the 15 conditional branches (Line 7 in Listing 4) is taken, and the backward branch to .LOOPENTRY is also taken. Following this trend, we build templates for 4-bit values from 0 to 14. Each case executes 17 branches: the first branch is not taken, the last branch is taken, and at most one of the branches in between is taken based on the 4-bit value. The pattern slightly differs when the 4-bit value is 15. As shown at .LABEL15, it takes the conditional branch at Line 17 and thus skips the direct branch at Line 19. Therefore, it executes 16 branches, and only the last branch is taken. We present the 16 patterns in Figure 4. For each execution of the Karatsuba base case, these patterns always appear 15 times continuously.

Identifying Patterns of Processing y . In HQC-128 decapsulation, Karatsuba base case is executed 9857 times and only 277 of them leak bits of y . Since the algorithm is deterministic, the positions of these cases are fixed and do not depend on the ciphertext or the secret key. Therefore, we learn their positions by simulating the decapsulation process and printing when the base case process y .

Practical Success Rate. We randomly generate 10 keys and repeat the attack 10 times for each key, resulting in 100 independent attacks. We confirm that we can always automatically recognize 9857 Karatsuba base cases and extract bits [63:4] of each word of y without errors.

Problematic Compiler Optimization Investigation. We further investigate which compiler induces exploitable leakages under which optimization level. GCC-13 does not induce key-dependent branches under any optimization level, while Clang-18 generates exploitable key-dependent branches under -O1, -O2, and -O3.

8. Detecting and Mitigating Timing Leakages

So far, we have shown that all code-based KEMs in the NIST R4 PQC submission are vulnerable to timing attacks. The mitigation of our attacks and all previous timing attacks on PQC KEMs is to write constant-time source code and ensure the binary is also constant-time. More than 50 tools have been published to assist writing constant-time code [71]. However, a user study [72] shows that they are rarely used in practice because of installation issues, poor documentation, poor usability, poor scalability, and so on.

To bridge the gap, we propose a workflow to develop constant-time cryptographic code, using HQC as an example, with CT-grind [35]. It only adds one line of code to the source code to taint a secret and only costs seconds to complete the analysis. With this workflow, we find all HQC timing leakages that have been exploited before and by us. We also analyze the new reference HQC implementation released after our initial responsible disclosure and find that it still exhibits compiler-induced timing leakages. To mitigate these leakages, we adopt the *volatile*-based technique to stop compilers from optimizing variables that are marked as *volatile* [73]. As expected, our workflow does not report constant-time violations after deploying the mitigation.

CT-grind. CT-grind [35] is the first constant-time analysis tool published in 2010, which is built upon Valgrind [74]. It covers all three types of constant-time violations: secret-dependent control-flow transfer, memory access, and variable-time operations (this feature is patched by [14]). It works by marking the memory that holds the secret as not initialized and performing dynamic taint analysis to track if the data in the uninitialized memory propagates to the aforementioned three operations. When compiling the code with debug information, CT-grind also reports the line of source code that is not constant-time. According to a user study [72], CT-grind is the most widely recognized tool among the interviewed cryptographic developers. It has already been used by several libraries such as BoringSSL (Link), PQClean (Link), and TIMECOP (Link).

Constant-Time Analysis Workflow. Using CT-grind requires the user to first identify all secrets. It is challenging for people who are not familiar with the library, but relatively easy for code developers. Therefore, we recommend that developers mark whether a function argument may process secrets to facilitate constant-time analysis. After knowing where the secret is, the first step of the workflow is to ❶ taint the memory area that holds the secret with the CT-grind API: `VALGRIND_MAKE_MEM_UNDEFINED`. It takes two parameters: the variable name and its length. Since CT-grind is a dynamic analysis tool, it requires an executable to carry out the analysis. Therefore, the second step is to ❷ build test cases to execute functions that process secrets. After preparing test cases, we need to compile the code and run tests. We recommend first analyzing whether the source code is constant-time by ❸ compiling the test cases with `-O0` optimization. After ensuring the source code is constant-time, the last step is to analyze if compilers can induce constant-time violations by ❹ compiling the test cases with different compilers (e.g., both `GCC` and `Clang`) and different optimizations. We recommend testing all common compiler optimization levels (e.g., `-O0` $\sim$ `-O3` and `-Os`). Depending on the threat model of whether an attacker can control the compilation process (e.g., by poisoning the `Makefile`), more compiler flags should be tested.

HQC Constant-Time Analysis. We now show how we apply the analysis workflow to the HQC PQClean implementation. This strategy can be adopted for all the other HQC implementations. All submitted KEMs in NIST PQC provide the same APIs for `KEM.Keygen`, `KEM.Encaps`,

and `KEM.Decaps`. Both `KEM.Keygen` and `KEM.Encaps` do not take secrets as input. Instead, they generate secrets internally. Therefore, we taint the secret randomnesses (`KEM.Keygen` randomnesses, `KEM.Encaps` randomness) inside them. `KEM.Decaps` is a deterministic function controlled by the two inputs: ciphertext and secret key. We taint the secret key but not the ciphertext, as it is usually considered public. We construct test cases by first invoking `KEM.Keygen` to randomly generate a key pair, then invoke `KEM.Encaps` to generate a ciphertext with them, and finally decapsulates it with `KEM.Decaps`. As CT-grind only works on addresses, it is not sensitive to the concrete processing values once all instructions can be executed. We compile the test case with both `GCC-13` and `Clang-18`, the default compilers provided by Ubuntu 24.04 TLS. After analyzing the constant-time property of source code with `-O0` optimization, we further test whether common optimization levels `-O1`, `-O2`, `-O3`, and `-Os` induce timing leakages.

Analysis Results. We analyze three HQC reference implementations, as well as the PQClean one, and confirm that CT-grind can detect all known timing vulnerabilities (those exploited by us and prior work [16], [17], [18]) within five seconds. We present the full analysis results in Table 2 in the Appendix A. Therefore, our results show that all timing attacks on HQC can be mitigated by properly analyzing the code with CT-grind when developing the implementation.

Latest HQC Reference Implementation. Besides the *Sparse Vector Hamming Weight Leakage*, the latest HQC reference implementation also has compiler-induced leakages in `find_peaks` (`Clang-18/GCC-13 -O0`. Link), `gf_carryless_mul` (`Clang-18 -O1`. Link 1 and Link 2), and `compute_error_values` (`Clang-18 -O2/-O3/-Os`. Link 1 and Link 2). Notably, the leakages in `find_peaks` have already been attacked by power side-channels to recover the secret key `y` [22]. `gf_carryless_mul` appears both in encapsulation and decapsulation, resulting in a potential encoded-message recovery to recover the session key (encapsulation), or a potential PC oracle to drive a CCA (decapsulation). `compute_error_values` may possibly reveal the number of errors corrected by the Reed-Solomon decoder to conduct a CCA similar to [26].

Discussion. Besides analyzing HQC, we also apply CT-grind to BIKE and Classic McEliece. We confirm that CT-grind can also spot the leakages we exploit in Section 6. Our experiments show that it is neither hard nor complex to detect constant-time violations. Therefore, we recommend that cryptographic code developers adopt CT-grind during the development cycle to mitigate timing attacks in advance.

9. Conclusion

In this paper, we provide a new perspective on evaluating the implementation security of code-based KEMs and exploiting their timing leakages. We identify and exploit two new timing leakages in the PQClean implementation of HQC. For the first time, we show that an attacker can recover the secret key of HQC without relying on a CCA. Instead, an attacker can exploit timing leakages arising from

the processing of sparse vectors, which do not depend on the ciphertext. Our new attacks are entirely passive and require only one measurement of the decapsulation of an arbitrary ciphertext. We show that these leakages and attacks apply generically and have low requirements on the attacker. In fact, our findings affect the implementations of all code-based KEMs that were submitted to Round 4 of the NIST standardization process and to both timing and power side-channel attacks. To help mitigate timing attacks during code development, we propose and evaluate a constant-time analysis workflow, demonstrating that CT-grind can effectively detect constant-time violations. Overall, the findings of this paper complement existing security analyses of code-based KEM implementations and assist developers in mitigating timing leakages during development.

References

- [1] P. W. Shor, “Algorithms for quantum computation: Discrete logarithms and factoring,” in *FOCS*, 1994, pp. 124–134. [Online]. Available: <https://doi.org/10.1109/SFCS.1994.365700>
- [2] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard,” Department of Commerce, Washington, D.C., Tech. Rep., 2023, federal Information Processing Standards Publication (FIPS) NIST FIPS 203 ipd. <https://doi.org/10.6028/NIST.FIPS.203.ipd>.
- [3] Gorjan Alagic and Gorjan Alagic and Daniel Apon and David Cooper and Quynh Dang and Thinh Dang and John Kelsey and Jacob Lichtinger and Yi-Kai Liu and Carl Miller and Dustin Moody and Rene Peralta and Ray Perlner and Angela Robinson and Daniel Smith-Tone, “Status report on the third round of the nist post-quantum cryptography standardization process,” National Institute of Standards and Technology, Department of Commerce, Washington, D.C., Tech. Rep., 2022, nIST IR 8413-upd1 <https://nvlpubs.nist.gov/nistpubs/ir/2022/NIST.IR.8413-upd1.pdf>.
- [4] P. Gaborit, C. Aguilar-Melchor, N. Aragon, S. Bettaieb, L. Bidoux, O. Blazy, J.-C. Deneuville, E. Persichetti, G. Zémor, J. Bos *et al.*, “Hamming quasi-cyclic (hqc),” *NIST Post-Quantum Standardization, 4th Round; National Institute of Standards and Technology: Gaithersburg, MD, USA*, 2025.
- [5] N. Aragon, P. Barreto, S. Bettaieb, L. Bidoux, O. Blazy, J.-C. Deneuville, P. Gaborit, S. Ghosh, S. Gueron, T. Güneysu *et al.*, “Bike: bit flipping key encapsulation,” 2022.
- [6] M. R. Albrecht, D. J. Bernstein, T. Chou, C. Cid, J. Gilcher, T. Lange, V. Maram, I. Von Maurich, R. Misoczki, R. Niederhagen *et al.*, “Classic mceliece: conservative code-based cryptography,” 2022.
- [7] M. Campagna, C. Costello, B. Hess, A. Hutchinson, A. Jalali, K. Karabina, B. Koziel, B. LaMacchia, P. Longa, M. Naehrig *et al.*, “Supersingular isogeny key encapsulation,” 2022.
- [8] W. Castryck and T. Decru, “An efficient key recovery attack on SIDH,” in *Advances in Cryptology – EUROCRYPT 2023, Part V*, ser. Lecture Notes in Computer Science, C. Hazay and M. Stam, Eds., vol. 14008. Lyon, France: Springer, Cham, Switzerland, Apr. 23–27, 2023, pp. 423–447.
- [9] Gorjan Alagic and Maxime Bros and Pierre Ciadoux and David Cooper and Quynh Dang and Thinh Dang and John Kelsey and Jacob Lichtinger and Yi-Kai Liu and Carl Miller and Dustin Moody and Rene Peralta and Ray Perlner and Angela Robinson and Hamilton Silberg and Daniel Smith-Tone and Noah Waller, “Status report on the fourth round of the nist post-quantum cryptography standardization process,” National Institute of Standards and Technology, Department of Commerce, Washington, D.C., Tech. Rep., 2025, nIST Internal Report NIST IR 8545 <https://nvlpubs.nist.gov/nistpubs/ir/2025/NIST.IR.8545.pdf>.
- [10] J. Howe, T. Prest, and D. Apon, “SoK: How (not) to design and implement post-quantum cryptography,” in *Topics in Cryptology – CT-RSA 2021*, ser. Lecture Notes in Computer Science, K. G. Paterson, Ed., vol. 12704. Virtual Event: Springer, Cham, Switzerland, May 17–20, 2021, pp. 444–477.
- [11] National Institute of Standards and Technology, “Submission requirements and evaluation criteria for the post-quantum cryptography standardization process,” National Institute of Standards and Technology, Department of Commerce, Washington, D.C., Tech. Rep., 2016, <https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/call-for-proposals-final-dec-2016.pdf>.
- [12] P. C. Kocher, “Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems,” in *Advances in Cryptology – CRYPTO ’96*, ser. Lecture Notes in Computer Science, N. Koblitz, Ed., vol. 1109. Santa Barbara, CA, USA: Springer Berlin Heidelberg, Germany, Aug. 18–22, 1996, pp. 104–113.
- [13] D. J. Bernstein, “Cache-timing attacks on aes,” 2005.
- [14] D. J. Bernstein, K. Bhargavan, S. Bhasin, A. Chattopadhyay, T. K. Chia, M. J. Kannwischer, F. Kiefer, T. B. Paiva, P. Ravi, and G. Tamvada, “Kyberslash: Exploiting secret-dependent division timings in kyber implementations,” *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2025, no. 2, pp. 209–234, 2025. [Online]. Available: <https://doi.org/10.46586/tches.v2025.i2.209-234>
- [15] —, “Kyberslash: Libraries,” <https://kyberslash.cr.yt.to/libraries.html>, 2025, version: 2025.08.23, accessed 2025-11-10.
- [16] Q. Guo, C. Hlauschek, T. Johansson, N. Lahr, A. Nilsson, and R. L. Schröder, “Don’t reject this: Key-recovery timing attacks due to rejection-sampling in HQC and BIKE,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2022, no. 3, pp. 223–263, 2022.
- [17] S. Huang, R. Q. Sim, C. Chuengsatiansup, Q. Guo, and T. Johansson, “Cache-timing attack against HQC,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2023, no. 3, pp. 136–163, 2023.
- [18] R. L. Schröder, S. Gast, and Q. Guo, “Divide and surrender: Exploiting variable division instruction timing in HQC key recovery attacks,” in *USENIX Security 2024: 33rd USENIX Security Symposium*, D. Balzarotti and W. Xu, Eds. Philadelphia, PA, USA: USENIX Association, Aug. 14–16, 2024. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/schr%C3%B6der>
- [19] M. J. Kannwischer, P. Schwabe, D. Stebila, and T. Wiggers, “Improving software quality in cryptography standardization projects,” in *EuroS&P 2022 - Workshops*. Los Alamitos, CA, USA: IEEE Computer Society, 2022, pp. 19–30. [Online]. Available: <https://eprint.iacr.org/2022/337>
- [20] P. Ravi, S. S. Roy, A. Chattopadhyay, and S. Bhasin, “Generic side-channel attacks on CCA-secure lattice-based PKE and KEMs,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2020, no. 3, pp. 307–335, 2020. [Online]. Available: <https://tches.iacr.org/index.php/TCHES/article/view/8592>
- [21] K. Xagawa, A. Ito, R. Ueno, J. Takahashi, and N. Homma, “Fault-injection attacks against NIST’s post-quantum cryptography round 3 KEM candidates,” in *Advances in Cryptology – ASIACRYPT 2021, Part II*, ser. Lecture Notes in Computer Science, M. Tibouchi and H. Wang, Eds., vol. 13091. Singapore: Springer, Cham, Switzerland, Dec. 6–10, 2021, pp. 33–61.
- [22] T. B. Paiva, P. Ravi, D. Jap, S. Bhasin, S. Das, and A. Chattopadhyay, “Et tu, brute? side-channel assisted chosen ciphertext attacks using valid ciphertexts on HQC KEM,” in *Post-Quantum Cryptography - 16th International Workshop, PQCrypto 2025, Taipei, Taiwan, April 8-10, 2025, Proceedings, Part II*, ser. Lecture Notes in Computer Science, R. Niederhagen and M. O. Saarinen, Eds., vol. 15578. Springer, 2025, pp. 294–321. [Online]. Available: https://doi.org/10.1007/978-3-031-86602-9_11

- [23] R. Lórencz, “Profiling side-channel attack on hqc polynomial multiplication using machine learning methods.”
- [24] H. Dong and Q. Guo, “Ot-pca: New key-recovery plaintext-checking oracle based side-channel attacks on hqc with offline templates,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2025, no. 1, pp. 251–274, 2025.
- [25] T. Schamberger, L. Holzbaur, J. Renner, A. Wachter-Zeh, and G. Sigl, “A power side-channel attack on the reed-muller reed-solomon version of the HQC cryptosystem,” in *Post-Quantum Cryptography - 13th International Workshop, PQCrypto 2022*, J. H. Cheon and T. Johansson, Eds. Virtual Event: Springer, Cham, Switzerland, Sep. 28–30, 2022, pp. 327–352.
- [26] G. Goy, A. Loiseau, and P. Gaborit, “A new key recovery side-channel attack on HQC with chosen ciphertext,” in *Post-Quantum Cryptography - 13th International Workshop, PQCrypto 2022*, J. H. Cheon and T. Johansson, Eds. Virtual Event: Springer, Cham, Switzerland, Sep. 28–30, 2022, pp. 353–371.
- [27] N. Maillet, C. Nugier, V. Migliore, and J.-C. Deneuville, “Key recovery from side-channel power analysis attacks on non-SIMD HQC decryption,” in *Advances in Cryptology – CRYPTO 2025, Part V*, ser. Lecture Notes in Computer Science, Y. T. Kalai and S. F. Kamara, Eds., vol. 16004. Santa Barbara, CA, USA: Springer, Cham, Switzerland, Aug. 17–21, 2025, pp. 70–102.
- [28] C. Baisse, A. Moran, G. Goy, J. Maillard, N. Aragon, P. Gaborit, M. Lecomte, and A. Loiseau, “Secret and shared keys recovery on hamming quasi-cyclic with sasca,” *Designs, Codes and Cryptography*, vol. 93, no. 6, pp. 2137–2157, 2025.
- [29] H. Dong and Q. Guo, “Multi-value plaintext-checking and full-decryption oracle-based attacks on hqc from offline templates,” *IACR Transactions on Cryptographic Hardware and Embedded Systems*, vol. 2025, no. 4, pp. 254–289, 2025.
- [30] Z. Huang, W. Wang, X. Zhou, and Y. Yu, “Dpa-style attacks on HQC,” *IACR Cryptol. ePrint Arch.*, p. 1966, 2025. [Online]. Available: <https://eprint.iacr.org/2025/1966>
- [31] H. Dong, Q. Guo, and D. Nabokov, “Single-trace key recovery attacks on HQC using valid and invalid ciphertexts,” *Cryptology ePrint Archive*, Paper 2025/1987, 2025. [Online]. Available: <https://eprint.iacr.org/2025/1987>
- [32] D. Lazard, “Gröbner bases, gaussian elimination and resolution of systems of algebraic equations,” in *European Conference on Computer Algebra*. Springer, 1983, pp. 146–156.
- [33] E. Prange, “The use of information sets in decoding cyclic codes,” *IRE Trans. Inf. Theory*, vol. 8, no. 5, pp. 5–9, 1962. [Online]. Available: <https://doi.org/10.1109/TIT.1962.1057777>
- [34] R. Zhang, A. Cheu, A. Gascón, D. Moghimi, P. Schoppmann, M. Schwarz, and O. Suci, “Farfetch’d: A side-channel analysis framework for privacy applications on confidential virtual machines,” *CoRR*, vol. abs/2506.15924, 2025. [Online]. Available: <https://doi.org/10.48550/arXiv.2506.15924>
- [35] A. Langley, “ctgrind: Checking that functions are constant time with valgrind,” <https://github.com/agl/ctgrind>, 2010, accessed: 2025-04-10.
- [36] National Institute of Standards and Technology, “Status report on the fourth round of the nist post-quantum cryptography standardization process,” Department of Commerce, Washington, D.C., Tech. Rep., 2025, NIST Internal Report NIST IR 8545 <https://doi.org/10.6028/NIST.IR.8545>.
- [37] E. Fujisaki and T. Okamoto, “Secure integration of asymmetric and symmetric encryption schemes,” in *Advances in Cryptology – CRYPTO ’99*, ser. Lecture Notes in Computer Science, M. J. Wiener, Ed., vol. 1666. Santa Barbara, CA, USA: Springer Berlin Heidelberg, Germany, Aug. 15–19, 1999, pp. 537–554.
- [38] —, “Secure integration of asymmetric and symmetric encryption schemes,” *Journal of Cryptology*, vol. 26, no. 1, pp. 80–101, Jan. 2013.
- [39] D. Hofheinz, K. Hövelmanns, and E. Kiltz, “A modular analysis of the Fujisaki-Okamoto transformation,” in *TCC 2017: 15th Theory of Cryptography Conference, Part I*, ser. Lecture Notes in Computer Science, Y. Kalai and L. Reyzin, Eds., vol. 10677. Baltimore, MD, USA: Springer, Cham, Switzerland, Nov. 12–15, 2017, pp. 341–371.
- [40] P. J. Lee and E. F. Brickell, “An observation on the security of McEliece’s public-key cryptosystem,” in *Advances in Cryptology – EUROCRYPT ’88*, ser. Lecture Notes in Computer Science, C. G. Günther, Ed., vol. 330. Davos, Switzerland: Springer Berlin Heidelberg, Germany, May 25–27, 1988, pp. 275–280.
- [41] J. Stern, “A method for finding codewords of small weight,” in *Coding Theory and Applications, 3rd International Colloquium, Toulon, France, November 2-4, 1988, Proceedings*, ser. Lecture Notes in Computer Science, G. D. Cohen and J. Wolfmann, Eds., vol. 388. Springer, 1988, pp. 106–113. [Online]. Available: <https://doi.org/10.1007/BFb0019850>
- [42] A. May, A. Meurer, and E. Thomae, “Decoding random linear codes in $\tilde{O}(2^{0.054n})$,” in *Advances in Cryptology – ASIACRYPT 2011*, ser. Lecture Notes in Computer Science, D. H. Lee and X. Wang, Eds., vol. 7073. Seoul, South Korea: Springer Berlin Heidelberg, Germany, Dec. 4–8, 2011, pp. 107–124.
- [43] A. Becker, A. Joux, A. May, and A. Meurer, “Decoding random binary linear codes in $2^{n/20}$: How $1 + 1 = 0$ improves information set decoding,” in *Advances in Cryptology – EUROCRYPT 2012*, ser. Lecture Notes in Computer Science, D. Pointcheval and T. Johansson, Eds., vol. 7237. Cambridge, UK: Springer Berlin Heidelberg, Germany, Apr. 15–19, 2012, pp. 520–536.
- [44] Q. Guo, T. Johansson, and V. Nguyen, “A new sieving-style information-set decoding algorithm,” *IEEE Trans. Inf. Theory*, vol. 70, no. 11, pp. 8303–8319, 2024. [Online]. Available: <https://doi.org/10.1109/TIT.2024.3457150>
- [45] P. Pessl and S. Mangard, “Enhancing side-channel analysis of binary-field multiplication with bit reliability,” in *Topics in Cryptology – CT-RSA 2016*, ser. Lecture Notes in Computer Science, K. Sako, Ed., vol. 9610. San Francisco, CA, USA: Springer, Cham, Switzerland, Feb. 29 – Mar. 4, 2016, pp. 255–270.
- [46] Q. Guo, T. Johansson, E. Mårtensson, and P. Stankovski, “Information set decoding with soft information and some cryptographic applications,” in *2017 IEEE International Symposium on Information Theory, ISIT 2017, Aachen, Germany, June 25-30, 2017*. IEEE, 2017, pp. 1793–1797. [Online]. Available: <https://doi.org/10.1109/ISIT.2017.8006838>
- [47] Q. Guo, T. Johansson, E. Mårtensson, and P. S. Wagner, “Some cryptanalytic and coding-theoretic applications of a soft stern algorithm,” *Adv. Math. Commun.*, vol. 13, no. 4, pp. 559–578, 2019. [Online]. Available: <https://doi.org/10.3934/amc.2019035>
- [48] N. Lahr, R. Niederhagen, R. Petri, and S. Samardjiska, “Side channel information set decoding using iterative chunking - plaintext recovery from the “classic McEliece” hardware reference implementation,” in *Advances in Cryptology – ASIACRYPT 2020, Part I*, ser. Lecture Notes in Computer Science, S. Moriai and H. Wang, Eds., vol. 12491. Daejeon, South Korea: Springer, Cham, Switzerland, Dec. 7–11, 2020, pp. 881–910.
- [49] A. Horlemann, S. Puchinger, J. Renner, T. Schamberger, and A. Wachter-Zeh, “Information-set decoding with hints,” in *Code-Based Cryptography - 9th International Workshop, CBCrypto 2021, Munich, Germany, June 21-22, 2021 Revised Selected Papers*, ser. Lecture Notes in Computer Science, A. Wachter-Zeh, H. Bartz, and G. Liva, Eds., vol. 13150. Springer, 2021, pp. 60–83. [Online]. Available: https://doi.org/10.1007/978-3-030-98365-9_4
- [50] S. Huang, R. Q. Sim, C. Chuengsatiansup, Q. Guo, and T. Johansson, “Cache-timing attack against HQC,” *Cryptology ePrint Archive*, Report 2023/102, 2023. [Online]. Available: <https://eprint.iacr.org/2023/102>

- [51] N. Veyrat-Charvillon, B. Gérard, and F.-X. Standaert, “Soft analytical side-channel attacks,” in *Advances in Cryptology – ASIACRYPT 2014, Part I*, ser. Lecture Notes in Computer Science, P. Sarkar and T. Iwata, Eds., vol. 8873. Kaoshiung, Taiwan, R.O.C.: Springer Berlin Heidelberg, Germany, Dec. 7–11, 2014, pp. 282–296.
- [52] “AMD64 Architecture Programmer’s Manual,” Advanced Micro Devices Inc., 2025.
- [53] A. Sev-Snp, “Strengthening vm isolation with integrity protection and more,” *White Paper*, January, 2020.
- [54] M. Li, L. Wilke, J. Wichelmann, T. Eisenbarth, R. Teodorescu, and Y. Zhang, “A systematic look at ciphertext side channels on AMD SEV-SNP,” in *43rd IEEE Symposium on Security and Privacy, SP 2022, San Francisco, CA, USA, May 22–26, 2022*. IEEE, 2022, pp. 337–351. [Online]. Available: <https://doi.org/10.1109/SP46214.2022.9833768>
- [55] L. Chiang and S. Li, “Reload+reload: Exploiting cache and memory contention side channel on AMD SEV,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2025, Rotterdam, Netherlands, 30 March 2025 - 3 April 2025*, L. Eeckhout, G. Smaragdakis, K. Liang, A. Sampson, M. A. Kim, and C. J. Rossbach, Eds. ACM, 2025, pp. 1014–1027. [Online]. Available: <https://doi.org/10.1145/3676641.3716017>
- [56] L. Giner, S. R. Neela, and D. Gruss, “Coherere+reload: Re-enabling high-resolution cache attacks on AMD SEV-SNP,” in *Detection of Intrusions and Malware, and Vulnerability Assessment - 22nd International Conference, DIMVA 2025, Graz, Austria, July 9–11, 2025, Proceedings, Part I*, ser. Lecture Notes in Computer Science, M. Egele, V. Moonsamy, D. Gruss, and M. Carminati, Eds., vol. 15747. Springer, 2025, pp. 191–212. [Online]. Available: https://doi.org/10.1007/978-3-031-97620-9_11
- [57] Y. Yuan, Z. Liu, S. Deng, Y. Chen, S. Wang, Y. Zhang, and Z. Su, “Ciphersteal: Stealing input data from tee-shielded neural networks with ciphertext side channels,” in *IEEE Symposium on Security and Privacy, SP 2025, San Francisco, CA, USA, May 12–15, 2025*, M. Blanton, W. Enck, and C. Nita-Rotaru, Eds. IEEE, 2025, pp. 4136–4154. [Online]. Available: <https://doi.org/10.1109/SP61157.2025.00079>
- [58] L. Wilke, J. Wichelmann, A. Rabich, and T. Eisenbarth, “Sev-step A single-stepping framework for AMD-SEV,” *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2024, no. 1, pp. 180–206, 2024. [Online]. Available: <https://doi.org/10.46586/tches.v2024.i1.180-206>
- [59] R. Zhang, L. Gerlach, D. Weber, L. Hetterich, Y. Lü, A. Kogler, and M. Schwarz, “Cachewarp: Software-based fault injection using selective state reset,” in *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14–16, 2024*, D. Balzarotti and W. Xu, Eds. USENIX Association, 2024. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/zhang-ruiyi>
- [60] B. Schlüter, S. Sridhara, A. Bertschi, and S. Shinde, “Wesee: Using malicious #vc interrupts to break AMD SEV-SNP,” in *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19–23, 2024*. IEEE, 2024, pp. 4220–4238. [Online]. Available: <https://doi.org/10.1109/SP54263.2024.00262>
- [61] B. Schlüter, S. Sridhara, M. Kuhne, A. Bertschi, and S. Shinde, “HECKLER: breaking confidential vms with malicious interrupts,” in *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14–16, 2024*, D. Balzarotti and W. Xu, Eds. USENIX Association, 2024. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/schl%C3%BCter>
- [62] N. Sendrier, “Secure sampling of constant-weight words – application to BIKE,” *Cryptology ePrint Archive*, Report 2021/1631, 2021. [Online]. Available: <https://eprint.iacr.org/2021/1631>
- [63] A. M. D. Inc., “Revision Guide for AMD Family 19h Models 10h–1Fh Processors,” 2025. [Online]. Available: https://docs.amd.com/v/u/en-US/57095-PUB_1_01
- [64] S. Gast, H. Weisstener, R. L. Schröder, and D. Gruss, “Counterseveillance: Performance-counter attacks on AMD SEV-SNP,” in *32nd Annual Network and Distributed System Security Symposium, NDSS 2025, San Diego, California, USA, February 24–28, 2025*. The Internet Society, 2025. [Online]. Available: <https://www.ndss-symposium.org/ndss-paper/counterseveillance-performance-counter-attacks-on-amd-sev-snp/>
- [65] J. Hermelink, K.-C. Ning, R. Petri, and E. Strieder, “The insecurity of masked comparisons: SCAs on ML-KEM’s FO-transform,” in *ACM CCS 2024: 31st Conference on Computer and Communications Security*, B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie, Eds. Salt Lake City, UT, USA: ACM Press, Oct. 14–18, 2024, pp. 2430–2444.
- [66] D. Hesse, J. Feldtkeller, T. Güneysu, J. Hermelink, G. Land, M. Krausz, and J. Richter-Brockmann, “t-probing (in-)security - pitfalls on noise assumptions,” *IACR Cryptol. ePrint Arch.*, p. 1202, 2025. [Online]. Available: <https://eprint.iacr.org/2025/1202>
- [67] Elaine Barker, “Recommendation for key management: Part 1 – general,” National Institute of Standards and Technology, Department of Commerce, Washington, D.C., Tech. Rep., 2020, nIST Special Publication 800-57 Part 1 Revision 5 <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57pt1r5.pdf>.
- [68] French Cybersecurity Agency (ANSSI), “Decision sur la securite algorithmique residuelle (remaining strength),” French Cybersecurity Agency (ANSSI), Tech. Rep., 2019, nOTE D’APPLICATION https://cyber.gouv.fr/sites/default/files/2022-08/anssi-cc-note-23-remaining-strength_v1.0%5B1%5D.pdf.
- [69] A. A. Karatsuba and Y. P. Ofman, “Multiplication of many-digital numbers by automatic computers,” in *Doklady Akademii Nauk*, vol. 145, no. 2, 1962, pp. 293–294.
- [70] R. P. Brent, P. Gaudry, E. Thomé, and P. Zimmermann, “Faster multiplication in $gf(2)[x]$,” in *Algorithmic Number Theory*, ser. Lecture Notes in Computer Science, vol. 5011, 2008, pp. 153–166. [Online]. Available: https://doi.org/10.1007/978-3-540-79456-1_10
- [71] Constant-timeness verification tools. [Online]. Available: <https://crocs-muni.github.io/ct-tools/>
- [72] J. Jancar, M. Fourné, D. D. A. Braga, M. Sabt, P. Schwabe, G. Barthe, P. Fouque, and Y. Acar, “they’re not that hard to mitigate”: What cryptographic library developers think about timing attacks,” in *SP*. IEEE, 2022, pp. 632–649. [Online]. Available: <https://doi.org/10.1109/SP46214.2022.9833713>
- [73] D. J. Bernstein, “The cryptoint library,” 2025. [Online]. Available: <https://cr.yp.to/papers.html#cryptoint>
- [74] N. Nethercote and J. Seward, “Valgrind: a framework for heavyweight dynamic binary instrumentation,” in *Proceedings of the ACM SIGPLAN 2007 Conference on Programming Language Design and Implementation, San Diego, California, USA, June 10–13, 2007*, J. Ferrante and K. S. McKinley, Eds. ACM, 2007, pp. 89–100. [Online]. Available: <https://doi.org/10.1145/1250734.1250746>

Latest HQC Algorithm

Algorithm 7 Latest HQC KEM.Keygen (Simplified)

Output: pk, sk

```

1: seedKEM ← Random()
2: seedPKE, σ ← Function(seedKEM)
3: ▷PKE.Keygen(seedPKE):
4:   (seedpk, seedsk) ←  $\mathcal{I}$ (seedPKE)
5:   ctxsk ← XOF.init(seedsk)
6:   - - x ← SampleFixedWeight§(ctxsk, ω)
7:   - - y ← SampleFixedWeight§(ctxsk, ω)
8:   ++ y ← SampleFixedWeight§(ctxsk, ω)
9:   ++ x ← SampleFixedWeight§(ctxsk, ω)
10:  ctxpk ← XOF.init(seedpk)
11:  h ← SampleVec(ctxpk)
12:  s ← x + h · y
13:  return pk = (seedpk, s), seedsk
14: return pk, sk = (pk, seedsk, σ, seedKEM)

```

Algorithm 8 Latest HQC KEM.Encaps (Simplified)

Input: pk
Output: ss, ct

```

1: (m, salt) ← Random()
2: (ss, θ) ←  $\mathcal{G}(\mathcal{H}(\text{pk}) || \text{m} || \text{salt})$ 
3: ▷PKE.Encrypt(pk, m, θ):
4:   seedpk, s ← pk
5:   ctxpk ← XOF.init(seedpk)
6:   h ← SampleVec(ctxpk)
7:   ctxθ ← XOF.init(seedθ)
8:   r2 ← SampleFixedWeight(ctxθ, ωr)
9:   e ← SampleFixedWeight(ctxθ, ωe)
10:  r1 ← SampleFixedWeight(ctxθ, ωr)
11:  u ← r1 + h · r2
12:  v ←  $\mathcal{C}.$ encode(m) + s · r2 + e
13:  return u, v
14: return ss, ct = (u, v, salt)

```

Algorithm 9 Latest HQC KEM.Decaps (Simplified)

Input: sk, ct
Output: ss

```

1: seedsk, pk, σ ← sk
2: ▷PKE.Decrypt(seedsk, ct):
3:   ctxsk ← XOF.init(seedsk)
4:   - - x ← SampleFixedWeight§(ctxsk, ω)
5:   y ← SampleFixedWeight§(ctxsk, ω)
6:   m ←  $\mathcal{C}.$ decode(v - u · y)
7:   return m
8: (ss, θ) ←  $\mathcal{G}(\mathcal{H}(\text{pk}) || \text{m} || \text{salt})$ 
9: ct' ← PKE.Encrypt(pk, m, θ)
10: if m = ⊥ ∨ ct ≠ ct':
11:   ss ←  $\mathcal{J}(\mathcal{H}(\text{pk}) || \sigma || \text{ct})$ 
12: return ss

```

TABLE 2. ANALYZING FOUR HQC REFERENCE IMPLEMENTATIONS ACROSS COMPILERS AND OPTIMIZATIONS.

Impl.	Compiler	Key Re-sampling (This Work)					Division [18]					Error Vector Re-sampling [16], [50]					Multiplication (This Work)				
		O0	O1	O2	O3	Os	O0	O1	O2	O3	Os	O0	O1	O2	O3	Os	O0	O1	O2	O3	Os
NIST R3	GCC	✓	✓	✓	✓	✓	○	○	○	○	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Clang	✓	✓	✓	✓	✓	✓	○	○	○	○	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
NIST R4	GCC	○	○	○	○	○	✓	✓	✓	✓	✓	○	○	○	○	○	✓	✓	✓	✓	✓
	Clang	○	✓	✓	✓	✓	✓	✓	✓	✓	✓	○	✓	✓	✓	✓	✓	✓	✓	✓	✓
PQClean	GCC	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
	Clang	○	✓	✓	✓	✓	○	○	○	○	○	○	✓	✓	✓	✓	○	✓	✓	✓	○
Latest	GCC	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○	○
	Clang	○	✓	✓	✓	✓	○	○	○	○	○	○	✓	✓	✓	✓	○	✓	○	○	○

We use ✓ to present if the vulnerability was reported by CT-grind. We use ○ to present if CT-grind does not report that the binary has such a vulnerability.